UTOMO Bank

Dokumentasi Sistem Android Firebase dan Back Office

Versi dokumentasi 14 Oktober 2026 | Project Firebase utomobank-ab445 | Package com.example.bankapputomo

Dokumen ini menjelaskan kondisi implementasi yang ada di source code saat ini. Isinya membedakan fitur yang sudah berjalan, fitur yang masih berupa simulasi lokal, data yang disimpan di Firebase, serta langkah yang harus dilakukan sebelum aplikasi dipakai sebagai layanan perbankan produksi.

Baca Bab 4 untuk memahami fungsi setiap fitur mobile, Bab 5 sampai Bab 7 untuk memahami Firebase dan aliran data, dan Bab 9 untuk memahami Transaction Report secara khusus. Bagian batasan produksi harus dibaca sebelum deployment publik.

1 Ringkasan Sistem

UTOMO Bank adalah aplikasi mobile banking Android native berbasis Kotlin dan XML yang dipasangkan dengan portal Back Office berbasis HTML, CSS, dan JavaScript ES Modules. Android memakai Firebase Authentication, Cloud Firestore, Realtime Database, dan penyimpanan lokal SharedPreferences melalui AppStorage. Back Office memakai Firebase Web SDK untuk Firestore dan Realtime Database REST sebagai fallback.

Sistem ini sudah cocok untuk prototype operasional, demonstrasi, dan pengujian alur end to end. Namun, beberapa transaksi eksternal masih disimulasikan di dalam aplikasi. Transfer antarbank, settlement QRIS, jaringan biller, jaringan telekomunikasi, dan ATM fisik belum terhubung ke switch perbankan atau gateway produksi.

Lapisan	Teknologi	Tanggung jawab
Mobile	Kotlin, XML, AndroidX, Material, Lottie	UI, validasi input, alur transaksi, kartu, report, local cache, dan navigasi.
Identity	Firebase Authentication	Email/password dan Google Sign-In; UID menjadi identitas utama account.
Document data	Cloud Firestore	Profil, kartu per user, ticket, message, notification, staff, rules, audit, dan mirror kartu global.
Realtime and legacy mirror	Realtime Database	Mirror transaksi, balances, profile, beneficiary, saving goals, users, dan status maintenance.
Back Office	Vanilla JS plus Firebase Web SDK	Dashboard, customer, cards, transactions, tickets, maintenance, staff, canned response, audit.
Local state	SharedPreferences plus Gson	Session, user-scoped balance, kartu, transaksi, currency ledger, goals, settings, dan fallback.

1.1 Teknologi Yang Digunakan

Bagian ini menjelaskan arti setiap teknologi dan alasan teknologi tersebut dipakai. Kotlin adalah bahasa pemrograman, XML adalah bahasa markup untuk layout, Firebase adalah platform backend, sedangkan AndroidX,

Material, Lottie, CameraX, Retrofit, dan Gson adalah library atau toolkit yang membantu aplikasi menjalankan fungsi tertentu.

Teknologi	Apa itu	Peran di UTOMO Bank
Kotlin	Bahasa pemrograman modern yang diketik secara statis dan memiliki null safety.	Menulis Activity, Fragment, model data, validasi, navigasi, transaksi, dan integrasi Firebase Android.
XML	Extensible Markup Language; pada Android dipakai sebagai bahasa deklaratif untuk mendeskripsikan tampilan.	Mendefinisikan layout screen, TextView, EditText, ImageView, tombol, padding, warna, dan struktur view. XML yang dimaksud di project ini bukan CML.
Android SDK	Kumpulan API, tools, emulator, dan build tools resmi Android.	Menyediakan Activity lifecycle, permission, Intent, resource system, manifest, dan proses pembuatan APK.
AndroidX	Library resmi Android yang meneruskan dan memperbaiki Android Support Library.	Menyediakan AppCompat, Core KTX, ConstraintLayout, ViewPager2, Biometric, SwipeRefreshLayout, dan komponen lifecycle.
Material Components	Komponen UI Android dengan pola interaksi dan aksesibilitas Material Design.	Dipakai untuk card, dialog, bottom sheet, button, input, dan komponen visual yang konsisten.
Gradle Kotlin DSL	Sistem build Android dengan konfigurasi Gradle yang ditulis menggunakan Kotlin pada file .gradle.kts.	Mengatur applicationId, SDK, dependency, plugin Google Services, dan konfigurasi build APK.
Firebase Authentication	Layanan identity as a service untuk login dan session.	Email/password dan Google Sign-In; UID Firebase menjadi identitas account di database.
Cloud Firestore	Database cloud berbasis dokumen dan collection dengan query serta realtime listener.	Menyimpan profil, kartu, ticket, message, notification, staff, audit, flags, dan maintenance.
Realtime Database	Database cloud berbentuk tree JSON dengan sinkronisasi realtime dan REST endpoint.	Menjadi mirror untuk balance, transaksi, profile, beneficiary, saving goals, users, dan maintenance.
SharedPreferences	Penyimpanan key-value lokal di perangkat Android.	Menyimpan session, preferensi, saldo user-scoped, daftar kartu, transaksi lokal, dan fallback development.
Gson	Library serialisasi dan deserialisasi object Kotlin atau Java ke JSON.	Mengubah daftar transaksi, kartu, goal, user, dan currency ledger menjadi data lokal yang dapat disimpan.
Lottie	Library untuk menjalankan animasi berbasis JSON.	Menampilkan animasi splash dan animasi login dari resource raw aplikasi.
CameraX	Library camera Android dengan API	Mendukung kamera QR atau scan flow

Teknologi	Apa itu	Peran di UTOMO Bank
	lifecycle yang lebih mudah dibanding camera API lama.	dan menangani lifecycle kamera pada device.
Retrofit dan Gson Converter	Retrofit adalah HTTP client type-safe; converter mengubah response JSON menjadi object.	Menjadi fondasi endpoint API development yang tersedia di project, walaupun settlement bank eksternal masih simulasi.
SwipeRefreshLayout	Container Android untuk gesture tarik ke bawah dan indikator refresh.	Dipakai pada dashboard agar user dapat memicu render ulang data setelah perubahan.
HTML CSS JavaScript	Teknologi dasar web untuk struktur, styling, dan perilaku halaman.	Membangun Back Office SPA tanpa framework besar atau bundler tambahan.
Firebase Web SDK	SDK JavaScript untuk mengakses layanan Firebase dari browser.	Menghubungkan admin panel ke Firestore, Authentication, dan Realtime Database REST fallback.
Node.js Cloud Functions	Runtime server-side JavaScript untuk menjalankan fungsi berbasis event atau callable.	Source tersedia untuk role management, notification, token revocation, dan automation; deployment production perlu diverifikasi terpisah.

1.1.1 Hubungan Kotlin Dan XML

Kotlin mengatur perilaku, sedangkan XML mengatur bentuk awal tampilan. Contohnya activity_login.xml mendefinisikan field email, password, tombol, dan container; LoginActivity.kt mencari view tersebut dengan findViewById, memasang click listener, memvalidasi input, lalu memanggil Firebase Auth. Karena keduanya bekerja bersama, perubahan id atau struktur XML harus disesuaikan dengan referensi Kotlin agar aplikasi tidak mengalami resource atau view lookup error.

1.1.2 Hubungan Firebase Dan AppStorage

Firebase menjadi sumber data cloud dan sinkronisasi antar perangkat, sedangkan AppStorage menjadi cache dan state lokal agar UI cepat serta tetap dapat menangani kondisi jaringan lambat. AppStorage bukan pengganti Firebase. Untuk data finansial produksi, saldo dan transaksi harus ditentukan oleh server yang tervalidasi, lalu local state hanya menyimpan cache tampilan atau state session.

1.1.3 Hubungan Mobile Dan Back Office

Aplikasi mobile menulis data customer melalui Firebase Android SDK. Back Office membaca collection yang sama melalui Firebase Web SDK. Karena keduanya berbicara ke backend yang sama, perubahan kartu, ticket, maintenance, atau audit dapat terlihat di kedua sisi. Perbedaan path atau penggunaan mirror RTDB dan Firestore harus dijaga konsisten agar panel tidak menampilkan data lama.

2 Struktur Aplikasi

Launcher activity adalah SplashActivity. Splash memeriksa status maintenance dan routing session sebelum mengarahkan user ke onboarding, login, atau dashboard. DashboardActivity menjadi pusat navigasi untuk home, history, statistics, settings, QRIS, dan quick services.

Kelompok	Activity atau komponen utama	Fungsi
Pembukaan	SplashActivity, OnboardingActivity	Brand splash, maintenance check, dan onboarding user baru.
Auth	LoginActivity, RegisterActivity	Email/password, Google Sign-In, biometric/PIN flow, dan pembuatan profile.
Home	DashboardActivity, AccountCardActivity, CardDetailActivity, CreditCardActivity	Saldo, kartu, quick action, detail kartu, blokir, dan kontrol keamanan.
Transaksi	TransferActivity, BillPaymentActivity, MobilePrepaidActivity, EWalletTopUpActivity, WithdrawActivity, ReceiveActivity	Alur transaksi yang mengurangi atau menambah saldo dan membuat history.
Valas	ExchangeActivity, ExchangeRateActivity	Konversi saldo antar currency dan tampilan saldo currency positif.
Analitik	StatisticsActivity, TransactionReportActivity, ReceiptActivity	Statistik enam bulan, kategori, batas pengeluaran, report bulanan, dan receipt.
Support	CustomerSupportActivity, SupportTicketsActivity, TicketDetailActivity	Ticket dan chat dua arah antara customer dan staff.
System	NotificationActivity, MaintenanceActivity, SuspendedAccountActivity, LanguageActivity	Notification, maintenance, suspend account, dan preferensi user.

3 Alur User Dari Buka Aplikasi

1. SplashActivity menampilkan logo dan animasi UTOMO dengan background putih.

2. Aplikasi membaca config/maintenance dari Realtime Database dan mirror Firestore. Jika enabled bernilai true dan user tidak memiliki bypass, user masuk ke MaintenanceActivity.

3. Jika maintenance tidak aktif, aplikasi membaca flag onboarding dan session login dari AppStorage.

4. User baru masuk onboarding lalu login atau register. User lama masuk dashboard jika session masih valid.

5. Saat login Firebase berhasil, UID dipakai untuk mengambil profil dan kartu. Jika data lokal sudah ada, data user-scoped dipertahankan agar saldo tidak kembali ke nilai awal.

6. Dashboard menampilkan saldo, kartu, quick service, transaksi terbaru, dan bottom navigation. Swipe refresh memuat ulang tampilan dari state yang tersedia.

4 Fitur Mobile Secara Mendalam

4.1 Dashboard dan Navigasi

Dashboard menampilkan saldo utama, kartu aktif, quick services, money pockets atau saving goals, promo, transaksi terbaru, kartu virtual, dan bottom navigation. Quick services mengarahkan user ke activity khusus, bukan menampilkan seluruh field input di dashboard. Layout memakai edge-to-edge handling dan area bawah disediakan agar bottom navigation tidak tertutup system navigation bar.

Swipe refresh digunakan untuk mengulang render data lokal dan memperbarui tampilan setelah transaksi, perubahan kartu, atau perubahan data dari listener Firebase. Swipe refresh bukan sinkronisasi server universal; setiap modul tetap bergantung pada repository atau endpoint yang dipakai modul tersebut.

4.2 Kartu dan Saldo

Setiap user memiliki kartu di users/{uid}/cards/{cardId}. Untuk kebutuhan Card Controls, kartu yang sama dimirror ke cards/{uid}_{cardId}. Nomor kartu dipakai untuk deduplikasi di panel agar satu kartu tidak tampil dua kali ketika terbaca dari koleksi global dan subkoleksi user.

Saldo debit disimpan menggunakan key user-scoped debit_balance_{uid}. Ketika transaksi berhasil, debitBalance berubah, balance pada kartu debit ikut diperbarui, lalu kartu dan saldo user disinkronkan ke Firestore. Saat login ulang atau instalasi baru, aplikasi mencoba memulihkan saldo dari record kartu Firestore sebelum membuat kartu awal.

- Logout tidak menghapus saldo scoped account.
- Login account berbeda tidak memakai saldo account sebelumnya.
- Kartu debit tersimpan di Firestore menjadi fallback jika user document masih berisi saldo awal.
- Panel Card Controls membaca global cards dan users/{uid}/cards lalu mendeduplikasi berdasarkan nomor kartu.

4.3 Transaksi dan History

TransactionModel menyimpan id, title, category, amount, income atau expense, counterparty, date, reference number, status, fee, recipient account, bank, note, origin, timestamp, dan userId bila tersedia. Transaksi baru diberi timestamp saat dibuat agar History dan Statistics dapat mengelompokkan transaksi berdasarkan waktu aktual.

Transaksi expense mengurangi debitBalance ketika parameter adjustDebitBalance aktif. Transaksi income menambah saldo. Beberapa operasi non-IDR hanya mengubah currency ledger dan mencatat aktivitas exchange tanpa mengurangi saldo IDR. Setelah transaksi, data lokal, mirror Realtime Database, Firestore transaction record, dan kartu debit diperbarui sesuai jalur yang dipakai activity.

4.4 Statistics

StatisticsActivity membangun statistik dari getTransactions(), bukan dari angka chart statis. Sistem membuat daftar enam bulan terakhir berdasarkan tanggal perangkat saat ini, menghitung pengeluaran sukses per bulan, menampilkan saldo, dan merender line, bar, atau pie chart.

- Month picker memilih satu bulan dan mengubah chart serta daftar transaksi terbaru.
- Spending limit tersimpan di AppStorage dan dibandingkan dengan pengeluaran bulan terpilih.
- Kategori dibagi ke food, entertainment, bills, dan lainnya melalui TransactionCategoryHelper.
- Saat activity resume, statistik dibangun ulang sehingga transaksi baru terlihat setelah kembali ke halaman.

4.5 Transaction Report

Transaction Report adalah halaman laporan atau e-statement untuk melihat dan membagikan ringkasan transaksi berdasarkan bulan. Tujuan bisnisnya adalah memberi user dokumen ringkas yang dapat disimpan atau dibagikan, berbeda dari Statistics yang fokus pada grafik dan analisis pengeluaran.

Bagian	Perilaku saat ini	Makna
Month selector	Memiliki pilihan Oktober 2026 sampai Mei 2026.	User memilih periode laporan yang ingin ditampilkan.
Report view	Area report dapat di-capture oleh activity.	Area ini menjadi sumber visual untuk file export.
Export	FileShareHelper.shareViewAsPdf membuat PDF dari view report.	PDF dapat dibagikan melalui Android share sheet.
Data dinamis	History dan Statistics sudah dinamis; Transaction Report belum melakukan query dan agregasi per bulan secara penuh.	Label bulan berubah, tetapi binding isi report perlu dilanjutkan jika e-statement real per bulan diperlukan.

Dengan demikian, Transaction Report saat ini sudah berfungsi sebagai UI pemilih bulan dan export view, tetapi belum boleh dianggap sebagai laporan bank final yang menjamin total income, total expense, kategori, dan daftar transaksi sudah terfilter sesuai bulan terpilih. Implementasi lanjutan yang disarankan adalah memakai fungsi agregasi yang sama dengan StatisticsActivity, lalu mengisi total dan rows report sebelum export.

4.6 Currency Exchange

Currency Exchange adalah fitur untuk menukar saldo antar mata uang yang sudah dimiliki account. Fitur ini bukan sekadar kalkulator kurs. Ia memvalidasi saldo sumber, menghitung nilai target, mengurangi saldo sumber, menambah saldo target, menyimpan ledger per account, dan membuat catatan aktivitas exchange.

Elemen	Penjelasan
From	Mata uang sumber yang akan dikurangi. Saldo dibaca dari currency ledger; IDR dibaca dari debitBalance.
To	Mata uang target yang akan bertambah setelah konversi berhasil.
Currency rate	Kurs silang dihitung dari ratesToUsd. Contoh source code: 1 USD = 15,650 IDR. Nilai ini adalah kurs aplikasi, bukan feed pasar langsung.
Balance validation	Exchange ditolak jika saldo From lebih kecil dari amount input atau From sama dengan To.
Per account	Ledger disimpan pada key currency_balances_{uid}, sehingga saldo currency user A tidak tercampur dengan user B.
Holdings	Di bawah currency rate ditampilkan hanya currency dengan nilai lebih besar dari nol. IDR juga ditampilkan karena berasal dari saldo utama account.
Audit activity	Exchange membuat transaction record dengan origin EXCHANGE dan counterparty UTOMO Bank Treasury FX atau UTOMO Bank FX.

Contoh: user menukar 100 USD menjadi IDR. Saldo USD berkurang 100, saldo IDR bertambah sesuai kurs, history mencatat aktivitas exchange, dan tampilan holdings memperlihatkan saldo currency yang masih lebih besar dari

nol. Jika menukar USD ke EUR belum tersedia karena daftar code saat ini adalah USD, IDR, KRW, VND, HKD, NTD, dan JMD.

Kurs saat ini bersifat hardcoded di ExchangeActivity. Untuk produksi, ratesToUsd sebaiknya diganti dengan endpoint rate yang tervalidasi, diberi timestamp, cache, batas usia data, dan mekanisme penolakan jika rate kedaluwarsa.

4.7 Budget Planner dan Saving Goals

Saving goals menyimpan nama target, target amount, current amount, deadline, warna atau icon, dan status progress. Dashboard menampilkan money pockets, sedangkan halaman input mempertahankan field dan tombol agar daftar goal dapat di-scroll tanpa menghilangkan kontrol utama. Monthly spending limit dipakai Statistics untuk menghitung persentase pengeluaran terhadap batas.

4.8 Favorite Transaction dan E Wallet

User dapat membuka shortcut penambahan transaksi favorit dari dashboard. User memilih tipe Bank atau E Wallet lalu diarahkan ke form yang sesuai. Data bank favorit mencakup bankName, recipientName, accountNumber, dan label terkait. Data e-wallet mencakup provider, nomor tujuan, nama pemilik, dan label. Setelah disimpan, shortcut muncul di halaman transaksi dan dashboard setelah refresh atau reload data.

4.9 Support Ticket dan Notification

CustomerSupportActivity membuat ticket pada tickets/{ticketId} dan message pada tickets/{ticketId}/messages/{messageId}. Pesan customer memakai senderType customer. Pesan staff dan auto-reply memakai senderType staff sehingga tidak salah tampil sebagai balasan customer. Listener Firestore memperbarui chat dan notifikasi tanpa user membuka ulang aplikasi.

5 Model Data Lokal

AppStorage adalah lapisan state lokal utama. Ia memakai SharedPreferences dan Gson. Local state membuat prototype tetap dapat dibuka ketika jaringan lambat, tetapi bukan pengganti database server. Key penting berikut sudah dibuat user-scoped untuk mencegah data account bercampur.

Key	Isi	Catatan
current_user_id	UID account aktif	Dipakai untuk semua key scoped.
debit_balance_{uid}	Saldo IDR utama	Dipakai kartu debit dan transaksi IDR.
savings_balance_{uid}	Saldo savings	Terpisah dari saldo debit utama.
currency_balances_{uid}	Map code currency ke amount	USD, KRW, VND, HKD, NTD, JMD; IDR dihitung dari debitBalance.
user_cards_json_{uid}	Daftar CardModel	Disinkronkan ke subkoleksi dan global cards.
transactions_json	Daftar TransactionModel lokal	Dipakai History dan Statistics; perlu dipastikan migrasi ke user scope untuk multi-account produksi.
registered_users_list	Fallback user lokal	Untuk development atau saat Firebase gagal; bukan sumber identity produksi.

6 Pemahaman Firebase

6.1 Project dan SDK

Project Firebase aktif adalah utomobank-ab445 dengan Realtime Database URL asia-southeast1. Android memakai google-services.json, plugin Google Services, Firebase BoM 33.7.0, Firebase Auth, Firebase Database, dan Firebase Firestore. Package Android adalah com.example.bankapputomo dan applicationId sama dengan package tersebut.

Firebase Authentication menentukan siapa usernya melalui UID. Database tidak seharusnya mengandalkan nama atau email sebagai primary key karena email dapat berubah. Firestore dan Realtime Database menyimpan data dengan pola berbeda dan pada project ini keduanya masih dipakai untuk fungsi yang berbeda serta beberapa mirror kompatibilitas.

6.2 Perbedaan Firestore dan Realtime Database

Layanan	Dipakai untuk	Cara aplikasi memakainya
Firebase Auth	Identity dan session	Email/password, Google Sign-In, UID, dan custom claims staff.
Cloud Firestore	Data dokumen terstruktur dan listener	users, cards, tickets, messages, notifications, staff, branches, audit, flags, maintenance.
Realtime Database	Mirror data sederhana dan status cepat	transactions, balances, user_profile, users, beneficiaries, saving_goals, config/maintenance.
Cloud Functions	Server-side automation	Source tersedia di functions, tetapi deployment dan fitur Blaze harus diperlakukan sebagai tahap opsional sampai diverifikasi.

Firestore unggul untuk query berbasis collection, subcollection, orderBy, whereEqualTo, dan snapshot listener. Realtime Database unggul untuk object tree sederhana dan REST read yang mudah dipakai fallback panel. Mengubah satu database tidak otomatis mengubah database lain; mirror harus ditulis secara eksplisit oleh code.

6.3 Struktur Firestore

Path	Isi dan tujuan
users/{uid}	Profil, email, phone, status, role, debitBalance, savingsBalance, google link, dan currencyBalances.
users/{uid}/cards/{cardId}	Kartu milik account. Sumber utama saat login ulang dan listener kartu.
cards/{uid}_{cardId}	Mirror global untuk Card Controls Back Office.
transactions/{txId}	Record immutable transaksi user; staff dapat membaca sesuai rules.
tickets/{ticketId}	Header ticket, status, priority, assignment, rating, dan timestamps.
tickets/{ticketId}/messages/{messageId}	Pesan customer, staff, atau auto-reply.

Path	Isi dan tujuan
tickets/{ticketId}/internalNotes/{noteId}	Catatan internal staff yang tidak boleh dibaca customer.
users/{uid}/notifications/{id}	Notifikasi card, ticket, account, dan system.
config/maintenance	enabled, startsAt, endsAt, message, updatedAt, dan metadata maintenance.
maintenance_bypass/{uid}	Daftar UID yang boleh melewati maintenance.
staff/{staffUid}	Profile staff dan role tambahan.
cannedResponses/{responseId}	Template jawaban support.
branches/{branchId}	Data branch dan ATM network.
transaction_flags/{txId}	Flag fraud atau review oleh staff.
audit_logs/{logId}	Log tindakan admin append-only.

6.4 Struktur Realtime Database

Path	Isi dan sifat data
users/{uid}	Mirror user untuk dibaca panel atau fallback.
user_profile	Profile legacy atau fallback global.
balances	Mirror saldo, credit limit, currencyBalances, dan updatedAt.
transactions	Mirror daftar transaksi untuk panel dan fallback.
beneficiaries	Daftar rekening atau transaksi favorit pada jalur legacy.
saving_goals	Mirror budget planner atau saving goals.
config/maintenance	Status maintenance yang dibaca Splash dan Login.

Karena beberapa node RTDB masih bersifat global, data itu harus dianggap sebagai mirror kompatibilitas, bukan model ideal multi-tenant. Untuk produksi, transaksi, beneficiaries, saving goals, dan balances sebaiknya diubah menjadi path per UID atau hanya dijadikan hasil server-side projection yang aman.

6.5 Alur Register Login dan Sinkronisasi

1. RegisterActivity membuat user di Firebase Auth dan mendapatkan UID.
2. Profile user ditulis ke Firestore users/{uid}; kartu awal ditulis ke users/{uid}/cards/card-1 dan mirror cards.
3. LoginActivity membaca credential atau Google result, lalu mengambil document user berdasarkan UID atau mapping account.
4. AppStorage.switchUser memilih key scoped UID. Jika ada saldo lokal, saldo tersebut dipertahankan; jika belum ada, nilai server atau saldo kartu dipakai sebagai initial value.
5. Perubahan balance menulis local state, mirror RTDB balances, dan Firestore user document. Perubahan card menulis subcollection dan global card mirror.

6. Back Office membaca Firestore terlebih dahulu dan memakai RTDB REST atau localStorage cache sebagai fallback demo saat Firestore tidak merespons.

6.6 Listener Real Time

Listener Firestore dipakai untuk maintenance, user status, ticket list, ticket messages, notifications, dan cards. Listener memberikan snapshot baru ketika dokumen berubah. Listener harus dilepas saat activity atau fragment dihancurkan agar tidak terjadi memory leak atau update ke view yang sudah tidak ada.

Maintenance memakai dua jalur: Splash membaca RTDB config/maintenance, sedangkan BackOfficeRepository juga memantau Firestore config/maintenance. Karena status dapat tertinggal bila hanya salah satu jalur diubah, panel menulis dan memverifikasi kedua mirror tersebut.

6.7 Security Rules dan Role

firestore.rules menggunakan Firebase Auth token dan custom claims role. Role yang dikenali adalah superadmin, support_agent, card_operator, dan viewer. Customer hanya boleh membaca atau membuat data yang terkait UID sendiri. Staff diberi akses sesuai fungsi, bukan akses penuh otomatis.

Area	Perlindungan yang diterapkan
Users	Customer membaca account sendiri; staff dapat membaca; customer tidak boleh mengubah status, role, atau balance melalui update profile.
Cards	Customer dapat freeze kartu sendiri pada kondisi yang diizinkan; staff dapat block; unblock dibatasi superadmin dan card_operator.
Transactions	Create hanya untuk user authenticated yang userId-nya sama; update dan delete ditolak.
Tickets	Customer hanya mengelola ticket sendiri; staff mengelola status, assignment, dan reply.
Internal notes	Hanya staff ticket yang boleh membaca atau menulis.
Audit logs	Staff dapat create dengan actorUid sendiri; update dan delete ditolak.
Maintenance	Read publik untuk status; write hanya superadmin; bypass berdasarkan UID.

Rules bukan pengganti validasi UI. UI dapat dibypass oleh client yang dimodifikasi, sehingga aturan penting harus tetap dipaksa oleh Firestore rules atau server-side function. Khusus balance dan transaksi, arsitektur produksi sebaiknya memakai server-authoritative transaction atau Cloud Function agar client tidak menjadi pihak yang menentukan saldo final.

7 Back Office

Back Office berada di folder admin-panel dan berjalan sebagai SPA tanpa bundler. index.html memuat app.js, styles.css, icons.js, dan firebase-config.js. Panel memiliki session timeout 30 menit, role guard di UI, Firestore Web SDK, RTDB REST fallback, dan localStorage cache untuk demo atau jaringan yang gagal.

Modul	Kegunaan

Modul	Kegunaan
Dashboard	KPI ticket, blocked cards, suspended users, flagged transactions, maintenance, audit terbaru.
Support Tickets	Queue ticket, canned response, reply staff, assignment, status, internal notes, audit.
Customers	Daftar customer, saldo, status, profile, suspend atau unsuspend.
Card Controls	All Customer Cards, search, block, freeze, unblock, staff hold, dan deduplikasi nomor kartu.
Transactions	Global Transactions Explorer dari RTDB dan Firestore, search, dan flag fraud.
Maintenance Mode	Mengaktifkan atau menonaktifkan maintenance dan menulis mirror RTDB serta Firestore.
Staff and Roles	Daftar staff dan role; provisioning claim tetap membutuhkan proses server-side.
Canned Responses	Template jawaban support yang dipakai agent.
Audit Trail	Riwayat tindakan administratif yang dibaca dari audit_logs.

8 Setup Firebase

8.1 Android

1. Pastikan google-services.json berasal dari project utomobank-ab445 dan diletakkan di folder app.

2. Pastikan package Android pada Firebase Android App sama dengan com.example.bankapputomo.

3. Untuk Google Sign-In, tambahkan SHA-1 dan SHA-256 debug serta release certificate di Firebase Project Settings, lalu download ulang google-services.json bila konfigurasi berubah.

4. Aktifkan Email/Password dan Google pada Authentication Sign-in method.

5. Buat Cloud Firestore dan Realtime Database pada project yang sama.

6. Sync Gradle dan jalankan aplikasi. Jika Google Sign-In gagal, periksa package, SHA certificate, OAuth client, dan projectId sebelum mengubah code.

```
implementation(platform("com.google.firebase:firebase-bom:33.7.0"))
implementation("com.google.firebase:firebase-auth")
implementation("com.google.firebase:firebase-database")
implementation("com.google.firebase:firebase-firestore")
```

8.2 Firestore dan Rules

1. Deploy rules dan index dari root project setelah meninjau perubahan rules.

2. Pastikan semua customer document memakai document id UID Firebase, bukan email atau nama.

3. Pastikan kartu memiliki userId pada global cards agar panel dan rules dapat menghubungkan kartu dengan account.

4. Uji read dan write menggunakan account customer, staff, dan viewer secara terpisah.

5. Jangan mengunduh atau menyimpan service account private key di APK, admin-panel publik, atau repository.

```
firebase login
firebase use utomobank-ab445
firebase deploy --only firestore:rules,firestore:indexes
```

8.3 Admin Panel

Jalankan start-admin-panel.bat dari root project. Script memilih port lokal yang tersedia dan membuka index.html pada port yang sesuai. Jangan membuka root project sebagai document root, karena browser dapat menampilkan Not Found jika URL tidak mengarah ke admin-panel/index.html.

Kredensial development yang tertanam di app.js hanya boleh dipakai lokal. Untuk deployment, login harus memakai Firebase Auth dan custom claims role yang benar.

9 Verifikasi Fitur

Uji	Langkah	Hasil yang diharapkan
Currency ledger	Login account A, exchange USD ke IDR, logout, login lagi.	USD berkurang, IDR bertambah, holdings positif tetap muncul untuk account A saja.
Account isolation	Login account B setelah account A.	Saldo, kartu, currency, dan profile B tidak memakai state A.
Card persistence	Transaksi dengan kartu, logout, login kembali.	Saldo kartu dan user record mempertahankan nilai terakhir.
Card Controls	Buat kartu atau update status kartu.	Panel menampilkan satu row per nomor kartu dan status sinkron.
Statistics	Buat transaksi baru lalu kembali ke Statistics.	Bulan, total pengeluaran, chart, kategori, dan recent transactions berubah.
Transaction Report	Pilih bulan lalu export.	Label bulan berubah dan PDF dibagikan; isi dinamis per bulan perlu diverifikasi sesuai batasan Bab 4.5.
Support	Customer buat ticket, staff reply.	Message staff tampil sebagai staff, bukan customer; notification masuk.
Maintenance	Admin ON lalu OFF.	Splash memblokir saat aktif dan aplikasi kembali normal setelah kedua mirror false.
Google login	Login pada APK release atau debug.	Berhasil jika provider aktif, package dan SHA certificate terdaftar.

10 Troubleshooting Firebase

Gejala	Kemungkinan penyebab	Pemeriksaan

Gejala	Kemungkinan penyebab	Pemeriksaan
Google Sign-In gagal	SHA-1/SHA-256 belum terdaftar atau google-services.json berbeda project.	Cocokkan package, projectId, OAuth client, dan certificate APK.
Maintenance tetap aktif	RTDB dan Firestore tidak sama, cache APK, atau project berbeda.	Periksa config/maintenance di dua layanan, nilai boolean enabled, device online, lalu clear app data untuk uji.
Saldo kembali 25 juta	User document lama atau kartu tidak terbaca.	Periksa users/{uid}.debitBalance, users/{uid}/cards, global cards.userId, dan log login.
Kartu dobel di panel	Kartu terbaca dari global dan nested collection atau cache lama.	Refresh panel, cek nomor kartu, userId, dan localStorage cache utomo_cards_cache.
Transaction Report tampak sama	Activity belum mengikat isi report ke query bulan.	Bedakan label month selector dengan data rows; implementasi agregasi per bulan diperlukan.
Panel blank atau Not Found	Server diarahkan ke root atau port berbeda.	Gunakan start-admin-panel.bat dan buka URL port yang dicetak script.
Permission denied	Custom claim belum ada atau token lama.	Set role server-side, sign out, sign in kembali, dan cek firestore.rules.

11 Batasan Produksi

- Kurs currency masih hardcoded dan belum merupakan market rate real time.
- Settlement transfer, QRIS, biller, telco, dan ATM masih simulasi lokal.
- Sebagian mirror Realtime Database masih global, sehingga perlu migrasi path per UID untuk isolasi multi-account yang kuat.
- Client masih memiliki fallback local seed dan localStorage cache. Fallback ini cocok untuk demo, bukan sumber kebenaran produksi.
- Manifest masih mengaktifkan usesCleartextTraffic dan source code memiliki endpoint development. Production harus memakai HTTPS dan endpoint yang dikonfigurasi aman.
- Kredensial development admin tidak boleh dipakai di deployment publik.
- Balance dan transaksi produksi sebaiknya diproses server-authoritative dengan Firestore transaction atau Cloud Functions.
- Cloud Functions tersedia di folder functions, tetapi aktivasi deployment, billing Blaze, dan uji trigger harus dilakukan terpisah.
- Transaction Report perlu binding agregasi per bulan sebelum disebut e-statement final.

12 Referensi Source Code

Area	File utama
Routing	app/src/main/java/com/example/bankapputomo/SplashActivity.kt
Auth	app/src/main/java/com/example/bankapputomo/LoginActivity.kt and RegisterActivity.kt

Area	File utama
Local state	app/src/main/java/com/example/bankapputomo/util/AppStorage.kt
RTDB mirror	app/src/main/java/com/example/bankapputomo/util/FirebaseSync.kt
Firestore and listeners	app/src/main/java/com/example/bankapputomo/util/BackOfficeRepository.kt
Statistics	app/src/main/java/com/example/bankapputomo/StatisticsActivity.kt
Transaction Report	app/src/main/java/com/example/bankapputomo/TransactionReportActivity.kt
Currency Exchange	app/src/main/java/com/example/bankapputomo/ExchangeActivity.kt
Admin panel	admin-panel/index.html, admin-panel/app.js, admin-panel/firebase-config.js
Security	firestore.rules, firestore.indexes.json, functions/index.js

Dokumen ini diperbarui berdasarkan source code dan konfigurasi project yang tersedia pada 14 Oktober 2026. Perubahan backend, Firebase Console, rules, atau struktur collection harus diikuti pembaruan dokumentasi agar uraian tetap sinkron dengan sistem.

Lampiran A Alur Kerja Dan Implementasi Teknis

Lampiran ini menjawab tujuh area yang diminta dengan pola tujuan, alur kerja, kode yang terlibat, teknologi, dan penyimpanan data. Penjelasan mengikuti source code yang tersedia di project pada 14 Oktober 2026. Istilah implemented berarti alurnya memang terlihat di source code; istilah prototype atau simulasi berarti UI dan alur tersedia, tetapi belum terhubung ke jaringan bank, biller, QRIS acquiring, atau server authoritative produksi.

Satu prinsip penting: Firebase Authentication menyimpan dan memverifikasi identitas login; Firestore dan Realtime Database menyimpan data aplikasi; AppStorage menyimpan cache serta state lokal perangkat. Ketiganya bukan hal yang sama dan tidak boleh dipertukarkan ketika aplikasi dikembangkan menjadi sistem produksi.

A.1 Login Dan Register

Tujuan Dan Perbedaan Firebase Authentication

Firebase Authentication adalah layanan Identity as a Service dari Firebase. Tugasnya adalah membuktikan bahwa seseorang benar-benar dapat masuk dengan metode tertentu, membuat user identity, menerbitkan token sesi, dan menyediakan UID unik. UID tersebut dipakai oleh Firestore Security Rules untuk menjawab pertanyaan: data ini milik user yang sedang login atau bukan? Firebase Authentication tidak otomatis menyimpan saldo, kartu, ticket, atau riwayat transaksi. Data bisnis itu harus disimpan di Firestore atau database lain secara terpisah.

Pada project ini metode yang digunakan adalah Email/Password dan Google Sign-In. Password seharusnya hanya masuk ke Firebase Authentication. Source saat ini masih menulis password ke dokumen Firestore dan local cache karena login lama berbasis prototype; pola itu tidak boleh dipakai untuk produksi karena password plaintext dapat membocorkan kredensial user.

Alur Register

1. RegisterActivity memvalidasi nama, KTP, email, nomor telepon, dan password pada form XML.

2. Aplikasi membuat identifier internal berbentuk user_email, membuat account number, lalu menyimpan RegisteredUserRecord ke AppStorage SharedPreferences menggunakan Gson.

3. Aplikasi membentuk userData dan mengirimkannya ke Realtime Database pada users/{uid} sebagai mirror serta ke Cloud Firestore pada users/{uid}.

4. Setelah Firestore selesai, aplikasi membuat kartu debit awal pada users/{uid}/cards/card-1 dan membuat mirror global pada cards/{uid}_card-1 untuk kebutuhan Card Controls di admin panel.

5. FirebaseAuth.createUserWithEmailAndPassword membuat identity Authentication. Setelah itu AppStorage.switchUser menetapkan user aktif dan membuka dashboard.

Potongan RegisterActivity.kt: membentuk profile, saldo awal, dan mirror RTDB.

```
private fun performRegistration(nama: String, ktp: String, email: String, phone: String, password:
String) {
        val uid = "user_" + email.replace(Regex("[^a-zA-Z0-9]"), "_")
        val nowStr = SimpleDateFormat("dd/MM/yyyy HH:mm:ss", Locale.getDefault()).format(Date())
        val accountNumber = "8801" + (100000..999999).random().toString()

        val storage = AppStorage.get(this)
        val userRecord = RegisteredUserRecord(
            uid = uid,
            name = nama,
            email = email,
            phone = phone,
            ktp = ktp,
            password = password,
            isPinSet = false,
            pin = "",
            biometricEnabled = false,
            googleLinked = false,
            googleEmail = "",
            debitBalance = 25000000L,
            savingsBalance = 0L,
            status = "active"
        )
        storage.saveRegisteredUser(userRecord)

        val userData = hashMapOf<String, Any>(
            "id" to uid,
            "uid" to uid,
            "name" to nama,
            "email" to email,
            "phone" to phone,
            "ktp" to ktp,
            "password" to password,
            "accountNumber" to accountNumber,
            "status" to "active",
            "role" to "customer",
            "googleLinked" to false,
            "googleEmail" to "",
            "pinEnabled" to false,
            "pin" to "",
            "biometricEnabled" to false,
            "debitBalance" to 25000000L,
            "savingsBalance" to 0L,
            "createdAt" to Timestamp.now(),
            "registeredAt" to nowStr
        )

        // Sync individual customer account to Firebase Realtime Database
        FirebaseSync.syncUserRecord(uid, userData)
```

Potongan RegisterActivity.kt: menyimpan users dan kartu ke Firestore serta membuat Auth identity.

```
// Save user to Firestore asynchronously
        try {
            BackOfficeRepository.firestore.collection("users").document(uid).set(userData)
                .addOnCompleteListener {
                    try {
                        val random15 = (1..15).map { (0..9).random() }.joinToString("")
                        val cardNumber = ("4" + random15).chunked(4).joinToString(" ")
                        val cardData = hashMapOf<String, Any>(
                            "id" to "card-1",
                            "userId" to uid,
                            "cardNumber" to cardNumber,
                            "cardHolder" to nama,
                            "expiry" to "12/29",
                            "balance" to 25000000L,
                            "cardType" to "Debit",
                            "gradientType" to 1,
                            "isBlocked" to false,
                            "status" to "active"
                        )
                        BackOfficeRepository.firestore.collection("users").document(uid)
                            .collection("cards").document("card-1").set(cardData)
                        // Keep a global mirror so the Admin Panel can manage
                        // every customer's card from one Card Controls page.
                        BackOfficeRepository.firestore.collection("cards")
                            .document("${uid}_card-1")
                            .set(cardData)
                    } catch (e: Exception) {}

                    try {
                        FirebaseAuth.getInstance().createUserWithEmailAndPassword(email, password)
                    } catch (e: Exception) {}

                    if (!regCompleted) {
                        regCompleted = true
                        regHandler.removeCallbacks(regTimeout)
                        completeRegistration(uid, nama, email, phone, accountNumber)
```

Alur Login Email Dan Password

LoginActivity lebih dulu mencari dokumen users berdasarkan email. Jika query email tidak menemukan dokumen, kode mencoba document ID user_email, kemudian mencoba local registered user. Setelah profile ditemukan, status akun diperiksa, password prototype dibandingkan, lalu establishFirebaseSession mencoba signInWithEmailAndPassword. Untuk akun lama yang hanya ada di Firestore, kode mencoba membuat identity Firebase Auth agar request Firestore memiliki request.auth.uid. Setelah berhasil, saveLoginAndProceed menyimpan session lokal dan mengarahkan user ke dashboard.

Potongan LoginActivity.kt: query profile, validasi status/password, dan pembentukan session Firebase.

```
private fun performFirebaseCredentialLogin(identifier: String, passwordEntered: String) {
        btnLogin.isEnabled = false
        btnLogin.alpha = 0.6f

        val targetEmail = if (identifier.contains("@")) identifier else
"$identifier@bankutomo.co.id"
        val docId = "user_" + targetEmail.replace(Regex("[^a-zA-Z0-9]"), "_")

        val storage = AppStorage.get(this)
        var loginHandled = false
        val loginHandler = Handler(Looper.getMainLooper())
        val loginTimeout = Runnable {
            if (!loginHandled) {
                loginHandled = true
                btnLogin.isEnabled = true
                btnLogin.alpha = 1.0f
                val localUser = storage.findRegisteredUserByEmail(targetEmail)
                if (localUser != null) {
                    validateLocalUser(localUser, passwordEntered)
                } else {
                    MaterialAlertDialogBuilder(this)
                        .setTitle("Akun Tidak Ditemukan")
                        .setMessage("Akun Anda belum terdaftar di Firebase. Silakan lakukan
registrasi terlebih dahulu.")
                        .setPositiveButton("Daftar Sekarang") { _, _ ->
                            startActivity(Intent(this, RegisterActivity::class.java))
                        }
                        .setNegativeButton("Tutup", null)
                        .show()
                }
            }
        }
        loginHandler.postDelayed(loginTimeout, 2000L)

        // First attempt to query by email in users collection
        BackOfficeRepository.firestore.collection("users")
            .whereEqualTo("email", targetEmail)
            .get()
            .addOnSuccessListener { snapshot ->
                if (!loginHandled) {
                    if (!snapshot.isEmpty) {
                        loginHandled = true
                        loginHandler.removeCallbacks(loginTimeout)
                        validateUserDocument(snapshot.documents[0], passwordEntered)
                    } else {
                        // Try doc ID directly
                        BackOfficeRepository.firestore.collection("users").document(docId).get()
                            .addOnSuccessListener { doc ->
                                if (!loginHandled) {
                                    loginHandled = true
                                    loginHandler.removeCallbacks(loginTimeout)
                                    if (doc.exists()) {
                                        validateUserDocument(doc, passwordEntered)
                                    } else {
                                        val localUser =
storage.findRegisteredUserByEmail(targetEmail)
                                        if (localUser != null) {
                                            validateLocalUser(localUser, passwordEntered)
                                        } else {
                                            btnLogin.isEnabled = true
```

```
                                        btnLogin.alpha = 1.0f
                                        MaterialAlertDialogBuilder(this)
                                            .setTitle("Akun Tidak Ditemukan")
                                            .setMessage("Akun Anda belum terdaftar di Firebase.
Silakan lakukan registrasi terlebih dahulu.")
                                            .setPositiveButton("Daftar Sekarang") { _, _ ->
                                                startActivity(Intent(this,
RegisterActivity::class.java))
                                            }
                                            .setNegativeButton("Tutup", null)
                                            .show()
                                    }
                                }
                            }
                        }
                        .addOnFailureListener {
                            if (!loginHandled) {
                                loginHandled = true
                                loginHandler.removeCallbacks(loginTimeout)
                                val localUser = storage.findRegisteredUserByEmail(targetEmail)
                                if (localUser != null) {
                                    validateLocalUser(localUser, passwordEntered)
                                } else {
                                    btnLogin.isEnabled = true
                                    btnLogin.alpha = 1.0f
                                    Toast.makeText(this, "Gagal menghubungi Firebase. Periksa
koneksi internet.", Toast.LENGTH_LONG).show()
                                }
                            }
                        }
                    }
                }
            }
            .addOnFailureListener {
                if (!loginHandled) {
                    loginHandled = true
                    loginHandler.removeCallbacks(loginTimeout)
                    val localUser = storage.findRegisteredUserByEmail(targetEmail)
                    if (localUser != null) {
                        validateLocalUser(localUser, passwordEntered)
                    } else {
                        btnLogin.isEnabled = true
                        btnLogin.alpha = 1.0f
                        Toast.makeText(this, "Gagal menghubungi Firebase. Periksa koneksi
internet.", Toast.LENGTH_LONG).show()
                    }
                }
            }
    }

    private fun validateUserDocument(doc: DocumentSnapshot, passwordEntered: String) {
        btnLogin.isEnabled = true
        btnLogin.alpha = 1.0f

        val status = doc.getString("status") ?: "active"
        if (status.equals("suspended", ignoreCase = true)) {
            val reason = doc.getString("statusReason") ?: "Akun dinonaktifkan oleh administrator."
            MaterialAlertDialogBuilder(this)
                .setTitle("Akun Ditangguhkan")
                .setMessage("Akun Anda ditangguhkan: $reason")
```

```
                .setPositiveButton("OK", null)
                .show()
            return
        }

        val storedPassword = doc.getString("password")
        if (storedPassword != null && storedPassword != passwordEntered) {
            containerPassword.setValidationState(false, iconPasswordValid, shakeOnError = true)
            Toast.makeText(this, "Kata sandi salah. Silakan coba lagi.", Toast.LENGTH_SHORT).show()
            return
        }

        establishFirebaseSession(doc, passwordEntered)
    }

    private fun establishFirebaseSession(doc: DocumentSnapshot, passwordEntered: String) {
        val email = doc.getString("email")?.trim().orEmpty()
        if (email.isEmpty()) {
            completeSuccessfulLogin(doc)
            return
        }

        val firebaseAuth = FirebaseAuth.getInstance()
        val currentUser = firebaseAuth.currentUser
        if (currentUser?.email.equals(email, ignoreCase = true)) {
            completeSuccessfulLogin(doc)
            return
        }

        firebaseAuth.signInWithEmailAndPassword(email, passwordEntered)
            .addOnSuccessListener {
                completeSuccessfulLogin(doc)
            }
            .addOnFailureListener {
                // Older accounts may exist in Firestore but not in Firebase Auth yet.
                // Create the Auth identity so Firestore security rules can identify it.
                firebaseAuth.createUserWithEmailAndPassword(email, passwordEntered)
                    .addOnSuccessListener {
                        completeSuccessfulLogin(doc)
                    }
                    .addOnFailureListener {
                        // Keep the existing prototype login flow even if Auth provisioning
                        // is unavailable; Firestore writes will be skipped until Auth exists.
                        completeSuccessfulLogin(doc)
                    }
            }
    }
```

Alur Login Google

Google Sign-In mengembalikan Google account dan ID token. ID token tidak langsung sama dengan session Firebase; token tersebut ditukar dengan GoogleAuthProvider.getCredential lalu dikirim ke FirebaseAuth.signInWithCredential. Setelah Firebase Auth selesai, aplikasi mencari profile berdasarkan email Google melalui checkAndLoginGoogleAccount. Jadi Google menyediakan bukti identitas, sedangkan Firebase Authentication menciptakan sesi yang dapat dikenali oleh Security Rules.

Potongan LoginActivity.kt: pertukaran Google ID token menjadi Firebase credential.

```
private val googleSignInLauncher =
registerForActivityResult(ActivityResultContracts.StartActivityForResult()) { result ->
        val task = GoogleSignIn.getSignedInAccountFromIntent(result.data)
        try {
            val account = task.getResult(ApiException::class.java)
            val email = account?.email ?: ""
            if (email.isNotEmpty()) {
                val idToken = account.idToken
                if (!idToken.isNullOrBlank()) {
                    FirebaseAuth.getInstance()
                        .signInWithCredential(GoogleAuthProvider.getCredential(idToken, null))
                        .addOnCompleteListener {
                            checkAndLoginGoogleAccount(email)
                        }
                } else {
                    checkAndLoginGoogleAccount(email)
                }
                return@registerForActivityResult
```

Data	Lokasi saat ini	Fungsi
Identity login	Firebase Authentication	Email/password, Google account, UID, token sesi.
Profile	Firestore users/{uid} dan RTDB users/{uid}	Nama, email, phone, account number, status, saldo mirror.
Session perangkat	AppStorage / SharedPreferences	is_logged_in, currentUserId, email, nama, preferensi.
Kartu awal	Firestore users/{uid}/cards/card-1 dan cards/{uid}_card-1	Kartu debit default dan sumber Card Controls.

A.2 QRIS Dan Aktivasi Kamera

Apa Yang Terjadi Saat Kamera Dibuka

QrFragment menggunakan Activity Result API untuk meminta Manifest.permission.CAMERA. Jika user memberi izin, startCamera mengambil ProcessCameraProvider, membuat Preview, menghubungkan preview ke PreviewView, memilih kamera belakang, melepas binding lama dengan unbindAll, lalu mengikat kamera ke lifecycle Fragment melalui bindToLifecycle. Karena lifecycle dipakai, kamera berhenti secara aman ketika Fragment tidak aktif.

Potongan QrFragment.kt: permission camera dan pemilih gambar dari gallery.

```
    private val cameraPermissionLauncher = registerForActivityResult(
            ActivityResultContracts.RequestPermission()
        ) { isGranted ->
            if (isGranted) {
                startCamera()
            } else {
                imgQrGraphic.visibility = View.VISIBLE
                cameraPreviewView.visibility = View.GONE
                Toast.makeText(requireContext(), "Camera permission denied. Tap frame to scan sample
QR.", Toast.LENGTH_SHORT).show()
            }
        }

        private val galleryPickerLauncher = registerForActivityResult(
            ActivityResultContracts.GetContent()
        ) { uri ->
            if (uri != null) {
                Toast.makeText(requireContext(), "QR Code detected from image!",
Toast.LENGTH_SHORT).show()
                simulateQrPayment("STARBUCKS SENAYAN CITY", 65000)
            }
        }
```

Potongan QrFragment.kt: CameraX Preview dan lifecycle binding.

```
    private fun startCamera() {
            val cameraProviderFuture = ProcessCameraProvider.getInstance(requireContext())
            cameraProviderFuture.addListener({
                try {
                    val cameraProvider = cameraProviderFuture.get()
                    val preview = Preview.Builder().build().also {
                        it.setSurfaceProvider(cameraPreviewView.surfaceProvider)
                    }
                    val cameraSelector = CameraSelector.DEFAULT_BACK_CAMERA
                    cameraProvider.unbindAll()
                    camera = cameraProvider.bindToLifecycle(viewLifecycleOwner, cameraSelector,
preview)
                    imgQrGraphic.visibility = View.GONE
                    cameraPreviewView.visibility = View.VISIBLE
                } catch (e: Exception) {
                    imgQrGraphic.visibility = View.VISIBLE
                    cameraPreviewView.visibility = View.GONE
                }
            }, ContextCompat.getMainExecutor(requireContext()))
        }
```

Teknologi	Arti	Peran
CameraX	Library kamera Android yang terintegrasi dengan lifecycle.	Menampilkan live camera preview melalui PreviewView.
Activity Result API	API modern untuk permission dan hasil aktivitas.	Meminta CAMERA dan membaca URI gambar dari gallery.
PreviewView	View khusus untuk menampilkan output CameraX.	Permukaan tampilan gambar kamera pada viewfinder.
Manifest permission	Deklarasi kemampuan aplikasi pada AndroidManifest.xml.	Menyatakan aplikasi membutuhkan akses kamera dan internet.

Batasan QR Decoder Saat Ini

Kamera pada source saat ini sudah dapat aktif sebagai preview, tetapi QrFragment belum melakukan decoding barcode atau QRIS secara nyata. Tap pada viewfinder, input manual, dan gambar gallery memanggil simulateQrPayment dengan merchant serta nominal contoh. Karena itu pesan QR detected dari gallery belum membuktikan bahwa isi QR benar-benar dibaca. Implementasi produksi perlu menambahkan barcode analyzer seperti ML Kit Barcode Scanning atau ZXing, membaca payload QRIS, memvalidasi merchant, menampilkan detail, lalu meminta konfirmasi sebelum membuat transaksi.

Potongan QrFragment.kt: simulasi pembayaran setelah konfirmasi dan pembuatan receipt.

```
    private fun simulateQrPayment(merchantName: String, amount: Long) {
            if
    (!com.example.bankapputomo.util.CardSecurityHelper.isPrimaryCardActive(requireContext())) {

    com.example.bankapputomo.util.CardSecurityHelper.showCardLockedBottomSheet(requireContext())
                return
            }

            val formatted = IdrFormatter.format(amount)
            com.example.bankapputomo.util.CustomConfirmationDialog.show(
                context = requireContext(),
                title = "Konfirmasi Bayar QRIS",
                message = "Merchant: $merchantName\nTotal: $formatted\nBiaya Layanan: Gratis (Rp
    0)\nSumber Dana: Utomo Debit (**** 7890)",
                positiveText = "Bayar $formatted",
                negativeText = "Batal",
                iconRes = R.drawable.ic_qr,
                isDestructive = false
            ) {
                val success = AppStorage.get().addTransaction(
                    title = "QRIS: $merchantName",
                    category = "QRIS",
                    amount = amount,
                    isIncome = false,
                    counterparty = merchantName
                )
                refreshBalance()

                if (success) {
                    ReceiptActivity.start(
                        context = requireContext(),
                        title = "Pembayaran QRIS Merchant",
                        category = "QRIS",
                        amount = amount,
                        fee = 0L,
                        counterparty = merchantName,
                        recipientAccount = "NMID: ID1020021982901",
                        bankName = "QRIS Standar Indonesia",
                        note = "Scan QRIS Merchant",
                        merchantDetails = "ID1020021982901 • Terminal A-01 (ASPI)",
                        origin = "QRIS",
                        customRefNo = "QRIS-${(10000000..99999999).random()}"
                    )
                } else {
                    Toast.makeText(requireContext(), "Saldo tidak mencukupi untuk pembayaran QRIS",
    Toast.LENGTH_LONG).show()
```

Setelah konfirmasi, AppStorage.addTransaction membuat TransactionModel, menambahkan transaksi di posisi paling baru, mengurangi debitBalance jika transaksi keluar, memperbarui saldo kartu debit, menyimpan local transaction list, mengirim mirror RTDB, dan memanggil sinkronisasi Firestore transaksi. Jika saldo tidak cukup atau kartu frozen/blocked, fungsi mengembalikan false.

A.3 Sistem Chat 24 Jam Dan Customer Service

Model Data

Chat customer service bukan satu dokumen besar berisi seluruh percakapan. Satu tiket dibuat di tickets/{ticketId}; setiap percakapan disimpan sebagai subcollection tickets/{ticketId}/messages/{messageId}. Bentuk ini membuat status tiket, assignment, priority, rating, dan isi pesan dapat dipisahkan. Catatan internal staf memakai tickets/{ticketId}/internalNotes dan dibatasi agar customer tidak dapat membacanya.

Alur Customer Membuat Ticket

1. CustomerSupportActivity meminta subject, category, dan initial message.

2. BackOfficeRepository.createTicket membuat ticket document dengan userId, status open, priority normal, createdAt, updatedAt, lastMessageAt, dan unreadForStaff.

3. Pesan pertama dibuat pada subcollection messages dengan senderType customer dan senderId user aktif.

4. Kode prototype menambahkan satu pesan otomatis dari system_bot dengan senderName Customer Service (Auto-Reply). Ini adalah auto-reply, bukan balasan staf manusia.

5. Listener Firestore mengurutkan messages berdasarkan createdAt ascending. Perubahan baru langsung merender ulang bubble chat.

Potongan BackOfficeRepository.kt: membuat ticket, pesan awal, dan auto-reply bot.

```
fun createTicket(
        context: Context,
        subject: String,
        category: String,
        initialMessage: String,
        onSuccess: (String) -> Unit,
        onError: (Exception) -> Unit
    ) {
        val uid = getEffectiveUserId(context)
        val storage = AppStorage.get(context)
        val ticketRef = firestore.collection("tickets").document()
        val now = Timestamp.now()

        val ticketData = hashMapOf(
            "userId" to uid,
            "subject" to subject,
            "category" to category,
            "priority" to "normal",
            "status" to "open",
            "createdAt" to now,
            "updatedAt" to now,
            "lastMessageAt" to now,
            "lastMessageBy" to "customer",
            "unreadForUser" to false,
            "unreadForStaff" to true,
            "customerLanguage" to storage.language
        )

        ticketRef.set(ticketData)
            .addOnSuccessListener {
                val messageRef = ticketRef.collection("messages").document()
                val messageData = hashMapOf(
                    "senderType" to "customer",
                    "senderId" to uid,
                    "senderName" to storage.userName.ifEmpty { "Customer" },
                    "text" to initialMessage,
                    "createdAt" to now
                )
                messageRef.set(messageData)
                    .addOnSuccessListener {
                        // Customer Service Auto-Reply Thank-You dialog
                        val autoReplyRef = ticketRef.collection("messages").document()
                        val autoReplyText = "Terima kasih telah mengisi tiket bantuan! 🙏\n\nTiket
Anda (#${ticketRef.id.take(8).uppercase()}) telah kami teruskan ke staf Customer Service. Mohon
menunggu sebentar, staf kami akan segera merespons percakapan ini."
                        val autoReplyData = hashMapOf(
                            "senderType" to "staff",
                            "senderId" to "system_bot",
                            "senderName" to "Customer Service (Auto-Reply)",
                            "text" to autoReplyText,
                            "createdAt" to Timestamp(now.seconds + 1, now.nanoseconds)
                        )
                        autoReplyRef.set(autoReplyData).addOnCompleteListener {
                            onSuccess(ticketRef.id)
                        }
                    }
                    .addOnFailureListener { e ->
                        onSuccess(ticketRef.id)
                    }
            }
```

```
            .addOnFailureListener { e ->
                onError(e)
            }
    }
```

Potongan BackOfficeRepository.kt: listener realtime dan pengiriman pesan customer.

```
fun listenToTicketMessages(
        ticketId: String,
        onMessagesChanged: (List<TicketMessageModel>) -> Unit
    ): ListenerRegistration? {
        return try {
            firestore.collection("tickets").document(ticketId)
                .collection("messages")
                .orderBy("createdAt", Query.Direction.ASCENDING)
                .addSnapshotListener { snapshot, error ->
                    if (error != null) {
                        Log.w(TAG, "Messages listener error: ${error.message}")
                        return@addSnapshotListener
                    }
                    val list = mutableListOf<TicketMessageModel>()
                    snapshot?.documents?.forEach { doc ->
                        val msg = doc.toObject(TicketMessageModel::class.java)
                        if (msg != null) {
                            msg.id = doc.id
                            list.add(msg)
                        }
                    }
                    onMessagesChanged(list)
                }
        } catch (e: Exception) {
            Log.w(TAG, "listenToTicketMessages exception: ${e.message}")
            null
        }
    }

    fun sendCustomerMessage(
        context: Context,
        ticketId: String,
        text: String,
        onSuccess: () -> Unit,
        onError: (Exception) -> Unit
    ) {
        val uid = getEffectiveUserId(context)
        val storage = AppStorage.get(context)
        val now = Timestamp.now()
        val ticketRef = firestore.collection("tickets").document(ticketId)

        val messageData = hashMapOf(
            "senderType" to "customer",
            "senderId" to uid,
            "senderName" to storage.userName.ifEmpty { "Customer" },
            "text" to text,
            "createdAt" to now
        )

        ticketRef.collection("messages").add(messageData)
            .addOnSuccessListener {
                ticketRef.update(
                    mapOf(
                        "lastMessageAt" to now,
                        "lastMessageBy" to "customer",
                        "unreadForStaff" to true,
                        "updatedAt" to now
                    )
                )
                onSuccess()
```

```
            }
            .addOnFailureListener { e ->
                onError(e)
            }
    }
```

Field	Contoh	Makna
tickets/{id}	userId, subject, status, priority	Header ticket dan status antrian.
messages/{id}	senderType, senderId, senderName, text, createdAt	Satu pesan dalam percakapan.
senderType	customer atau staff	Membedakan pengirim dan gaya bubble.
system_bot	senderId system_bot	Auto-reply awal yang dibuat aplikasi.
internalNotes/{id}	Catatan staf	Informasi internal yang tidak dibaca customer.

Admin panel membaca ticket dan messages dengan Firebase Web SDK. Balasan manusia harus dikirim dari akun staff yang memiliki role support_agent atau superadmin. Firestore Rules memeriksa senderType, senderId, ukuran text, dan role sehingga customer tidak dapat berpura-pura sebagai staff.

A.4 Maintenance Dan Admin Panel

Alur Login Admin

Admin panel adalah SPA berbasis HTML, CSS, dan JavaScript ES Modules. Form login memanggil signInWithEmailAndPassword dari Firebase Web SDK. onAuthStateChanged kemudian mengambil Custom Claims melalui getIdTokenResult(user, true). Jika claim role belum ada, panel mencoba membaca staff/{uid}. Role valid saat ini adalah superadmin, support_agent, card_operator, dan viewer. Sidebar lalu menyembunyikan menu yang tidak sesuai izin.

Potongan admin-panel/app.js: login Firebase, refresh Custom Claims, role check, dan session guard.

```
// Login form submit
  const loginForm = document.getElementById('loginForm');
  loginForm.onsubmit = async (e) => {
    e.preventDefault();
    const email = document.getElementById('loginEmail').value.trim();
    const pass = document.getElementById('loginPassword').value;
    const errBox = document.getElementById('loginError');
    const submitBtn = document.getElementById('btnLoginSubmit');

    errBox.style.display = 'none';
    submitBtn.disabled = true;
    submitBtn.textContent = 'Authenticating...';

    try {
      await signInWithEmailAndPassword(auth, email, pass);
    } catch (err) {
      const isLocalDevAdmin = (email.toLowerCase() === LOCAL_DEV_CREDENTIALS.email.toLowerCase() &&
pass === LOCAL_DEV_CREDENTIALS.password);
      const isAuthOffline = (err.message && (err.message.toLowerCase().includes('api-key') ||
err.message.toLowerCase().includes('api key') || err.message.includes('network') ||
err.message.includes('internal')));

      if (isLocalDevAdmin || (email.toLowerCase() === LOCAL_DEV_CREDENTIALS.email.toLowerCase() &&
isAuthOffline)) {
        console.info('Activating back-office session for:', email);
        state.currentUser = { uid: 'demo-superadmin', email: LOCAL_DEV_CREDENTIALS.email,
displayName: LOCAL_DEV_CREDENTIALS.displayName };
        state.currentRole = LOCAL_DEV_CREDENTIALS.role;
        document.getElementById('userName').textContent = LOCAL_DEV_CREDENTIALS.displayName;
        document.getElementById('userRole').textContent = LOCAL_DEV_CREDENTIALS.role;
        document.getElementById('userAvatar').textContent =
LOCAL_DEV_CREDENTIALS.displayName.charAt(0);
        document.getElementById('loginContainer').style.display = 'none';
        document.getElementById('appContainer').style.display = 'flex';
        startInactivityObserver();
        navigateTo('dashboard');
        return;
      }
      errBox.textContent = `Sign in failed: ${err.message || 'Email atau password tidak sesuai.'}`;
      errBox.style.display = 'block';
      submitBtn.disabled = false;
      submitBtn.textContent = 'Sign In to Console';
    }
  };

  document.getElementById('btnSignOut').onclick = () => {
    signOut(auth);
  };

  // Auth State Listener
  onAuthStateChanged(auth, async (user) => {
    const loginContainer = document.getElementById('loginContainer');
    const appContainer = document.getElementById('appContainer');

    if (!user) {
      state.currentUser = null;
      state.currentRole = null;
      clearInterval(state.inactivityTimer);
      clearActiveListeners();
      loginContainer.style.display = 'flex';
```

```
      appContainer.style.display = 'none';
      return;
    }

    try {
      // 1. Check Custom Claims in Token
      const tokenResult = await getIdTokenResult(user, true);
      let role = tokenResult.claims.role;

      // 2. Spark tier fallback: inspect staff/{uid} document if claim is pending
      if (!role) {
        const staffDoc = await getDoc(doc(db, 'staff', user.uid));
        if (staffDoc.exists() && staffDoc.data().active) {
          role = staffDoc.data().role;
        }
      }

      const validRoles = ['superadmin', 'support_agent', 'card_operator', 'viewer'];
      if (!role || !validRoles.includes(role)) {
        alert('Access Denied: This account has not been provisioned with an authorized bank staff
role.');
        await signOut(auth);
        return;
      }

      state.currentUser = user;
      state.currentRole = role;

      // Update sidebar profile
      document.getElementById('userName').textContent = user.displayName ||
user.email.split('@')[0];
      document.getElementById('userRole').textContent = role.replace('_', ' ');
      document.getElementById('userAvatar').textContent = (user.displayName ||
user.email)[0].toUpperCase();

      // Enforce sidebar permissions
      navItems.forEach(item => {
        const el = document.getElementById(item.id);
        if (el) {
          if (item.roleReq && !item.roleReq()) {
            el.style.display = 'none';
          } else {
            el.style.display = 'flex';
          }
        }
      });

      loginContainer.style.display = 'none';
      appContainer.style.display = 'flex';
      startInactivityObserver();
      navigateTo('dashboard');
    } catch (err) {
      alert(`Staff verification error: ${err.message}`);
      await signOut(auth);
    }
  });
```

Alur Maintenance

Pada implementasi aktual, source admin panel menyimpan config maintenance ke Realtime Database melalui REST PUT pada endpoint /config/maintenance.json. Android BackOfficeRepository mendengarkan RTDB path config/maintenance lebih dulu dan memiliki fallback pembacaan Firestore pada config/maintenance. Ketika status aktif, Splash atau alur pemeriksaan maintenance mengarahkan user ke MaintenanceActivity. Tombol retry memanggil listener lagi; jika status sudah nonaktif, aplikasi kembali ke LoginActivity atau DashboardActivity sesuai session lokal.

Potongan MaintenanceActivity.kt: countdown, retry, dan routing setelah maintenance selesai.

```
override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_maintenance)

        textTitle = findViewById(R.id.textMaintenanceTitle)
        textMessage = findViewById(R.id.textMaintenanceMessage)
        layoutCountdown = findViewById(R.id.layoutMaintenanceCountdown)
        textCountdown = findViewById(R.id.textMaintenanceCountdown)
        btnRetry = findViewById(R.id.btnRetryMaintenance)
        btnSupport = findViewById(R.id.btnMaintenanceSupport)
        lottieAnim = findViewById(R.id.lottieMaintenanceAnim)

        // Initialize 404 Lottie Animation
        lottieAnim?.let {
            it.setAnimation(R.raw.anim_maintenance_404)
            LottieMotionHelper.playLoop(it)
        }

        val intentTitle = intent.getStringExtra("title")
        val intentMessage = intent.getStringExtra("message")
        val endsAt = intent.getLongExtra("endsAt", -1L)

        val defaultTitle = getString(R.string.maintenance_title_default)
        val defaultMessage = getString(R.string.maintenance_desc_default)

        textTitle.text = if (!intentTitle.isNullOrEmpty()) {
            if (!intentTitle.contains("404")) "404 - $intentTitle" else intentTitle
        } else {
            defaultTitle
        }

        textMessage.text = if (!intentMessage.isNullOrEmpty()) {
            intentMessage
        } else {
            defaultMessage
        }

        if (endsAt > System.currentTimeMillis()) {
            layoutCountdown.visibility = View.VISIBLE
            startCountdown(endsAt - System.currentTimeMillis())
        } else {
            layoutCountdown.visibility = View.GONE
        }

        btnRetry.bounceClick {
            checkAndProceed()
        }

        btnSupport?.bounceClick {
            val intent = Intent(this, CustomerSupportActivity::class.java)
            startActivity(intent)
        }

        onBackPressedDispatcher.addCallback(this, object : OnBackPressedCallback(true) {
            override fun handleOnBackPressed() {
                moveTaskToBack(true)
            }
        })
    }
```

```
    private fun startCountdown(millisRemaining: Long) {
        countDownTimer?.cancel()
        countDownTimer = object : CountDownTimer(millisRemaining, 1000) {
            override fun onTick(millisUntilFinished: Long) {
                val hours = (millisUntilFinished / (1000 * 60 * 60))
                val minutes = (millisUntilFinished / (1000 * 60)) % 60
                val seconds = (millisUntilFinished / 1000) % 60
                textCountdown.text = String.format(Locale.getDefault(), "%02d:%02d:%02d", hours,
minutes, seconds)
            }

            override fun onFinish() {
                layoutCountdown.visibility = View.GONE
                checkAndProceed()
            }
        }.start()
    }

    private fun checkAndProceed() {
        btnRetry.isEnabled = false
        btnRetry.text = getString(R.string.str_mohon_tunggu)

        BackOfficeRepository.listenToMaintenanceMode(this) { isUnderMaintenance, title, message,
endsAt ->
            btnRetry.isEnabled = true
            btnRetry.text = getString(R.string.maintenance_try_again)

            if (!isUnderMaintenance) {
                Toast.makeText(this, "Layanan perbankan telah kembali normal.",
Toast.LENGTH_SHORT).show()
                val targetActivity = if (AppStorage.get(this).isLoggedIn) {
                    DashboardActivity::class.java
                } else {
                    LoginActivity::class.java
                }
                val intent = Intent(this, targetActivity).apply {
                    flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
                }
                startActivity(intent)
                finish()
            } else {
                if (title.isNotEmpty()) {
                    textTitle.text = if (!title.contains("404")) "404 - $title" else title
                }
```

Potongan admin-panel/app.js: endpoint RTDB yang menerima config maintenance.

```
const MAINTENANCE_RTDB_URL = 'https://utomobank-ab445-default-rtdb.asia-
southeast1.firebasedatabase.app/config/maintenance.json';

async function persistMaintenanceState(maintenance) {
  const response = await fetch(MAINTENANCE_RTDB_URL, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(maintenance)
  });

  if (!response.ok) {
    throw new Error(`Realtime Database returned HTTP ${response.status}`);
  }

  const verified = await response.json();
  if (!verified || verified.enabled !== Boolean(maintenance.enabled)) {
```

Potongan admin-panel/app.js: toggle maintenance dan maintenance_bypass.

```
requireReason: true,
      onConfirm: async (reason) => {
        try {
          const nextMaint = { ...maint, enabled: willEnable };
          await persistMaintenanceState(nextMaint);
          maint = nextMaint;
          localStorage.setItem('utomo_maint_cache', JSON.stringify(maint));
          await logAudit(willEnable ? 'ENABLE_MAINTENANCE' : 'DISABLE_MAINTENANCE',
'system_config', 'maintenance', null, maint, reason);
          showToast(willEnable ? 'Mode pemeliharaan telah DIAKTIFKAN.' : 'Mode pemeliharaan telah
DINONAKTIFKAN.');
          renderMaintenance(container);
        } catch (error) {
          showToast(`Gagal menyimpan status maintenance: ${error.message}`, 'error');
        }
      }
    });
  };

  document.getElementById('btnSaveMaintContent').onclick = async () => {
    maint.title = {
      en: document.getElementById('maintTitleEn').value.trim(),
      id: document.getElementById('maintTitleId').value.trim()
    };
    maint.message = {
      en: document.getElementById('maintMsgEn').value.trim(),
      id: document.getElementById('maintMsgId').value.trim()
    };
    try {
      await persistMaintenanceState(maint);
      localStorage.setItem('utomo_maint_cache', JSON.stringify(maint));
      await logAudit('UPDATE_MAINTENANCE_NOTICE', 'system_config', 'maintenance', null, maint,
'Updated localized downtime text');
      showToast('Pesan downtime berhasil disimpan!');
    } catch (error) {
      showToast(`Gagal menyimpan pesan maintenance: ${error.message}`, 'error');
    }
  };

  document.getElementById('btnAddBypassUid').onclick = async () => {
    const uid = document.getElementById('inputNewBypassUid').value.trim();
    if (!uid) return;
    bypassList.push({ uid, addedAt: new Date().toISOString() });
    localStorage.setItem('utomo_bypass_cache', JSON.stringify(bypassList));
    showToast(`Tester UID "${uid}" berhasil dimasukkan ke whitelist bypass!`);
    renderMaintenance(container);
    try {
      await withTimeout(setDoc(doc(db, 'maintenance_bypass', uid), { active: true, addedAt:
serverTimestamp() }), 500);
    } catch (_) {}
  };
}

window.removeBypass = async (uid) => {
  const cached = localStorage.getItem('utomo_bypass_cache');
  let list = cached ? JSON.parse(cached) : [];
  list = list.filter(b => b.uid !== uid);
  localStorage.setItem('utomo_bypass_cache', JSON.stringify(list));
  showToast(`UID "${uid}" berhasil dihapus dari whitelist.`);
  renderMaintenance(document.getElementById('appContent'));
```

```
try {
  await withTimeout(deleteDoc(doc(db, 'maintenance_bypass', uid)), 500);
} catch (_) {}
```

Timeout Dan Fallback

withTimeout adalah helper Promise.race yang membatasi waktu tunggu operasi Firebase. Jika Firebase tidak menjawab dalam batas waktu, panel beralih ke cache localStorage seperti utomo_maint_cache, utomo_staff_cache, utomo_tickets_cache, dan cache modul lain. Cache membuat demo tetap terlihat, tetapi cache bukan bukti bahwa data sudah tersimpan di Firebase. Untuk operasi perbankan produksi, panel harus menampilkan status sinkronisasi dan tidak menganggap fallback lokal sebagai write sukses.

Potongan admin-panel/app.js: definisi withTimeout berbasis Promise.race.

```
// ============================================================
// TIMEOUT WRAPPER & DEFAULT ENTERPRISE DATASETS
// Mencegah halaman hang/beku saat Firestore offline atau tidak terjangkau
// ============================================================
export const withTimeout = (p, ms = 700) => Promise.race([
  p,
  new Promise((_, reject) => setTimeout(() => reject(new Error('timeout')), ms))
]);
```

Tempat Penyimpanan Maintenance Dan Bypass

Data	Lokasi	Penjelasan
Status maintenance	RTDB /config/maintenance	Sumber yang ditulis oleh toggle admin panel dan dibaca mobile lebih dulu.
Fallback status	Firestore config/maintenance	Cadangan pembacaan atau kompatibilitas; harus dijaga sama dengan RTDB.
Tester bypass	Firestore maintenance_bypass/{uid}	Dokumen dengan active dan addedAt; dicocokkan dengan UID Firebase Auth.
Cache browser	localStorage utomo_maint_cache dan utomo_bypass_cache	Cache tampilan atau fallback lokal, bukan sumber cloud utama.
Audit perubahan	Firestore audit_logs/{logId}	Catatan append-only ketika staf mengaktifkan, mematikan, atau mengubah konfigurasi.

Firebase Security Rules memeriksa request.auth, custom claim role, dan dokumen maintenance_bypass. Artinya bypass yang aman harus berisi UID Firebase Auth yang sama dengan user di perangkat; email atau user ID lokal saja tidak cukup.

A.5 Share Receipt

Tujuan Dan Alur

ReceiptActivity menampilkan bukti transaksi dari Intent extras atau TransactionModel. Data yang dibinding meliputi nominal, fee, status, tanggal, reference number, sender, recipient, rekening tujuan, bank, note, dan detail QRIS. Reference number dapat disalin ke clipboard. Area receipt dapat dibagikan sebagai gambar, PDF, atau disimpan ke gallery.

1. Activity transaksi memanggil ReceiptActivity.start dengan title, category, amount, fee, counterparty, bankName, note, origin, dan reference number.

2. ReceiptActivity.bindData membaca extras, memformat nominal dengan IdrFormatter, lalu mengisi layout dan animasi status sukses/gagal.

3. FileShareHelper.captureViewToBitmap menggambar View receipt ke Bitmap.

4. FileShareHelper.shareBitmap membuat file cache melalui FileProvider dan mengirim Intent ACTION_SEND dengan URI content yang memiliki FLAG_GRANT_READ_URI_PERMISSION.

5. Untuk PDF, Android PdfDocument menggambar view ke halaman PDF, menulis file ke cache/docs, lalu membagikannya lewat FileProvider.

Potongan ReceiptActivity.kt: tombol share image, share PDF, save gallery, dan navigasi transaksi berikutnya.

```
    textReceiptBreakdownTotal.text = IdrFormatter.format(total)
    }

    private fun setupListeners() {
        findViewById<View>(R.id.btnReceiptBack).bounceClick {
            finish()
        }

        findViewById<View>(R.id.btnReceiptCopyRef).bounceClick {
            val clipboard = getSystemService(Context.CLIPBOARD_SERVICE) as ClipboardManager
            clipboard.setPrimaryClip(ClipData.newPlainText("No Referensi", referenceNo))
            Toast.makeText(this, "Nomor referensi disalin", Toast.LENGTH_SHORT).show()
        }

        // 1. Share as Image
        findViewById<View>(R.id.btnShareReceiptImage).bounceClick {
            val bitmap = FileShareHelper.captureViewToBitmap(cardReceiptCaptureArea)
            FileShareHelper.shareBitmap(this, bitmap, "Bagikan Bukti Transaksi")
        }

        // 2. Share as PDF
        findViewById<View>(R.id.btnShareReceiptPdf).bounceClick {
            FileShareHelper.shareViewAsPdf(this, cardReceiptCaptureArea, "Bagikan Bukti PDF")
        }

        // 3. Save to Gallery
        findViewById<View>(R.id.btnSaveReceiptGallery).bounceClick {
            val bitmap = FileShareHelper.captureViewToBitmap(cardReceiptCaptureArea)
            FileShareHelper.saveBitmapToGallery(this, bitmap, "BankUtomo_Receipt")
        }

        // 4. Selesai (Done)
        findViewById<View>(R.id.btnReceiptDone).bounceClick {
            finish()
        }

        // 5. Transaksi Lagi (Make another transaction)
        val btnAnother = findViewById<TextView>(R.id.btnReceiptAnother)
        btnAnother.bounceClick {
            when (originFeature) {
                "TRANSFER" -> startActivity(Intent(this, TransferActivity::class.java))
                "BILL" -> startActivity(Intent(this, BillPaymentActivity::class.java))
                "PULSA" -> startActivity(Intent(this, MobilePrepaidActivity::class.java))
                "WITHDRAW" -> startActivity(Intent(this, WithdrawActivity::class.java))
                "SAVE" -> startActivity(Intent(this, SaveOnlineActivity::class.java))
                "EXCHANGE" -> startActivity(Intent(this, ExchangeActivity::class.java))
                else -> finish()
            }
            finish()
        }
    }
}
```

Potongan FileShareHelper.kt: capture View menjadi Bitmap dan membagikannya melalui FileProvider.

```
object FileShareHelper {

    /**
     * Renders any Android View hierarchy to a Bitmap with its current measurement.
     */
    fun captureViewToBitmap(view: View): Bitmap {
        val width = if (view.width > 0) view.width else view.measuredWidth
        val height = if (view.height > 0) view.height else view.measuredHeight

        val bitmap = Bitmap.createBitmap(
            width.coerceAtLeast(1),
            height.coerceAtLeast(1),
            Bitmap.Config.ARGB_8888
        )
        val canvas = Canvas(bitmap)
        canvas.drawColor(Color.TRANSPARENT)
        view.draw(canvas)
        return bitmap
    }

    /**
     * Shares a rendered bitmap as a PNG through Android's system share sheet using FileProvider.
     */
    fun shareBitmap(activity: Activity, bitmap: Bitmap, chooserTitle: String = "Bagikan Bukti
Transaksi") {
        try {
            val imagesFolder = File(activity.cacheDir, "images")
            imagesFolder.mkdirs()
            val file = File(imagesFolder, "BankUtomo_TRX_${System.currentTimeMillis()}.png")
            FileOutputStream(file).use { out ->
                bitmap.compress(Bitmap.CompressFormat.PNG, 100, out)
            }

            val uri = FileProvider.getUriForFile(
                activity,
                "${activity.packageName}.provider",
                file
            )

            val shareIntent = Intent(Intent.ACTION_SEND).apply {
                type = "image/png"
                putExtra(Intent.EXTRA_STREAM, uri)
                putExtra(Intent.EXTRA_SUBJECT, "Bukti Transaksi UTOMO Bank")
                addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
            }

            activity.startActivity(Intent.createChooser(shareIntent, chooserTitle))
        } catch (e: Exception) {
            e.printStackTrace()
            Toast.makeText(activity, "Gagal membagikan gambar: ${e.localizedMessage}",
Toast.LENGTH_SHORT).show()
        }
    }

    /**
     * Saves a rendered bitmap to the user's gallery in Pictures/BankUtomo.
     */
    fun saveBitmapToGallery(context: Context, bitmap: Bitmap, filenamePrefix: String =
"BankUtomo_Receipt"): Boolean {
```

Teknologi	Apa itu	Pemakaian
Intent ACTION_SEND	Mekanisme Android untuk menyerahkan konten ke aplikasi lain.	Membuka chooser WhatsApp, email, Drive, dan aplikasi berbagi lain.
FileProvider	ContentProvider aman untuk membagikan file tanpa file:// URI.	Memberikan content URI sementara ke image atau PDF receipt.
Bitmap	Representasi gambar di memori.	Hasil capture area receipt sebelum disimpan atau dibagikan.
PdfDocument	API Android untuk membuat PDF secara programatik.	Menggambar receipt ke satu halaman PDF.

Receipt yang dibagikan berasal dari data Intent dan layout lokal. Share receipt tidak otomatis membuat dokumen baru di Firestore atau Cloud Storage. Jika bank memerlukan arsip e-receipt, server perlu membuat immutable receipt record dan menyimpan file melalui Cloud Storage dengan reference transaction ID.

A.6 Card Dan Card Slider

Model Data Dan Persistensi

Model kartu berada di AppStorage sebagai CardModel. Data lokal diserialisasi Gson ke SharedPreferences dengan key user-scoped sehingga kartu Haris tidak bercampur dengan kartu user lain pada perangkat yang sama. Cloud copy berada di users/{uid}/cards/{cardId}; global mirror cards/{uid}_{cardId} dipakai admin panel agar seluruh kartu dapat ditampilkan dalam satu Card Controls page.

Cara Slider Bekerja

CardDeckHelper mengambil getCards dari AppStorage, menyaring kartu active dan tidak blocked, lalu menghapus isi deckContainer dan meng-inflate item_deck_card untuk maksimal tiga kartu. Kartu diurutkan dari belakang ke depan. Kartu belakang diberi translationY, scale sedikit lebih kecil, dan elevation berbeda sehingga tampak seperti stack. Kartu depan menerima touch/click; kartu lain memanggil cycleCardForward. Saat status kartu berubah atau listener Firestore menerima update, HomeFragment memanggil renderDeck ulang.

Potongan CardDeckHelper.kt: filter kartu, maksimal tiga layer, scale, translation, elevation, dan click handling.

```
class CardDeckHelper(
    private val activity: Activity,
    private val deckContainer: ViewGroup,
    private val onCardChanged: ((CardModel?) -> Unit)? = null
) {

    private val inflater: LayoutInflater = LayoutInflater.from(activity)
    private val density: Float = activity.resources.displayMetrics.density
    private var isAnimating = false

    fun setup(onlyActiveCards: Boolean = true) {
        renderDeck(animateFromBack = false, onlyActiveCards = onlyActiveCards)
    }

    fun renderDeck(animateFromBack: Boolean = false, onlyActiveCards: Boolean = true) {
        val allCards = AppStorage.get().getCards()
        val cards = if (onlyActiveCards) {
            allCards.filter { (it.status == "active" || it.status.isEmpty()) && !it.isBlocked }
        } else {
            allCards
        }
        deckContainer.removeAllViews()

        if (cards.isEmpty()) {
            onCardChanged?.invoke(null)
            return
        }

        onCardChanged?.invoke(cards.first())

        // Show up to 3 cards in the visible stack (Picture 5 style)
        val visibleCount = kotlin.math.min(cards.size, 3)
        // Add from back to front so front is drawn on top
        for (i in (visibleCount - 1) downTo 0) {
            val card = cards[i]
            val cardView = inflater.inflate(R.layout.item_deck_card, deckContainer, false)
            bindCardView(cardView, card)

            val targetTranslationY = (i * 16 * density)
            val targetScale = 1.0f - (i * 0.05f)
            val targetElevation = ((visibleCount - i) * 6 * density)

            cardView.pivotY = 0f
            cardView.scaleX = targetScale
            cardView.scaleY = targetScale
            cardView.elevation = targetElevation
            cardView.findViewById<View>(R.id.cardBodyContainer)?.elevation = targetElevation

            if (animateFromBack && i == visibleCount - 1) {
                cardView.translationY = -60f * density
                cardView.alpha = 0.5f
                cardView.animate()
                    .translationY(targetTranslationY)
                    .alpha(1.0f)
                    .setDuration(220)
                    .start()
            } else if (animateFromBack && i == 0) {
                cardView.translationY = 16f * density
                cardView.scaleX = 0.95f
                cardView.scaleY = 0.95f
```

```
                cardView.animate()
                    .translationY(0f)
                    .scaleX(1.0f)
                    .scaleY(1.0f)
                    .setDuration(220)
                    .start()
            } else {
                cardView.translationY = targetTranslationY
            }

            if (i == 0) {
                setupFrontCardTouchAndClick(cardView, card)
            } else {
                cardView.setOnClickListener {
                    cycleCardForward()
                }
            }

            deckContainer.addView(cardView)
        }
    }
```

Potongan HomeFragment.kt: tombol tambah/blokir/hapus dan listener realtime kartu.

```
    private fun setupCardDeck(view: View) {
        val deckContainer = view.findViewById<ViewGroup>(R.id.homeDeckContainer) ?: return
        cardDeckHelper = CardDeckHelper(requireActivity(), deckContainer)

        view.findViewById<View>(R.id.btnDeckAddCard)?.bounceClick {
            cardDeckHelper?.showAddCardBottomSheet()
        }

        view.findViewById<View>(R.id.btnDeckToggleBlock)?.bounceClick {
            cardDeckHelper?.blockCurrentCard()
        }

        view.findViewById<View>(R.id.btnDeckDeleteCard)?.bounceClick {
            cardDeckHelper?.deleteCurrentCard()
        }

        cardDeckHelper?.setup(onlyActiveCards = true)

        // Real-time listener for cards: if staff blocks or freezes a card, update Home deck
    immediately
        BackOfficeRepository.listenToUserCards(requireContext()) {
            if (isAdded) {
                cardDeckHelper?.renderDeck(animateFromBack = true, onlyActiveCards = true)
            }
        }
```

Aksi	Local state	Cloud state
Tambah kartu	AppStorage.addCard lalu saveCards.	users/{uid}/cards/{id} dan cards/{uid}_{id}.
Blokir atau freeze	CardModel status dan isBlocked berubah.	Card document diperbarui sesuai Security Rules.
Saldo transaksi	debitBalance dan balance kartu debit	users/{uid} balance fields dan kartu

Aksi	Local state	Cloud state
	berubah.	dimirror oleh repository.
Card slider	Tidak menyimpan posisi visual sebagai data bisnis.	Membaca daftar kartu terbaru dari local/cloud state.

Deduplikasi global penting karena kartu dibuat di subcollection user dan root cards. Document ID global berbentuk uid_cardId mencegah satu kartu yang sama masuk dua kali ke tabel admin. Admin panel tetap perlu menggabungkan data berdasarkan userId dan card id, bukan hanya nama cardholder.

A.7 Onboarding Pada Home Site

Dua Jenis Onboarding

Project memiliki onboarding pertama kali dan in-app guidance setelah user masuk home. Onboarding pertama kali adalah OnboardingActivity dengan ViewPager2 dan tiga slide drawable. In-app guidance memakai InAppGuidanceManager untuk menyorot greeting, card deck, quick action, dan tombol QRIS. Keduanya berbeda: OnboardingActivity mengantar user sebelum login; guidance menjelaskan home setelah dashboard tersedia.

Alur Onboarding Pertama Kali

1. SplashActivity memeriksa flag hasSeenOnboarding atau is_first_launch dari AppStorage.

2. Jika user baru, Intent menuju OnboardingActivity.

3. OnboardingActivity memasang OnboardingAdapter pada ViewPager2 dengan img_onboard_slide1 sampai img_onboard_slide3.

4. Tombol Next menaikkan currentItem; tombol Skip langsung menuju login; pada slide terakhir tombol register dan login ditampilkan.

5. Saat memilih register atau login, hasSeenOnboarding disimpan true sehingga instalasi berikutnya tidak mengulang layar onboarding kecuali session/reset mengubah flag tersebut.

Potongan OnboardingActivity.kt: slides, ViewPager2, tombol Next/Skip, dan penyimpanan flag.

```
private val slides = listOf(
        OnboardingItem(
            R.drawable.img_onboard_slide1,
            R.string.onboarding_slide_one_title,
            R.string.onboarding_slide_one_subtitle
        ),
        OnboardingItem(
            R.drawable.img_onboard_slide2,
            R.string.onboarding_slide_two_title,
            R.string.onboarding_slide_two_subtitle
        ),
        OnboardingItem(
            R.drawable.img_onboard_slide3,
            R.string.onboarding_slide_three_title,
            R.string.onboarding_slide_three_subtitle
        )
    )

    private lateinit var pager: ViewPager2
    private lateinit var btnNext: TextView
    private lateinit var btnSkip: TextView
    private lateinit var layoutAuthButtons: View
    private lateinit var btnRegister: TextView
    private lateinit var btnLogin: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_onboarding)

        pager = findViewById(R.id.pagerOnboarding)
        btnNext = findViewById(R.id.btnNextOnboarding)
        btnSkip = findViewById(R.id.btnSkipOnboarding)
        layoutAuthButtons = findViewById(R.id.layoutAuthButtons)
        btnRegister = findViewById(R.id.btnRegisterOnboarding)
        btnLogin = findViewById(R.id.btnLoginOnboarding)

        pager.adapter = OnboardingAdapter(slides)
        pager.registerOnPageChangeCallback(object : ViewPager2.OnPageChangeCallback() {
            override fun onPageSelected(position: Int) {
                super.onPageSelected(position)
                if (position == slides.size - 1) {
                    btnNext.visibility = View.GONE
                    btnSkip.visibility = View.GONE
                    layoutAuthButtons.visibility = View.VISIBLE
                } else {
                    btnNext.visibility = View.VISIBLE
                    btnSkip.visibility = View.VISIBLE
                    layoutAuthButtons.visibility = View.GONE
                }
            }
        })

        btnNext.bounceClick {
            if (pager.currentItem < slides.size - 1) {
                pager.currentItem += 1
            }
        }

        btnSkip.bounceClick {
            proceedToLogin()
```

```
        }

        btnRegister.bounceClick {
            proceedToRegister()
        }

        btnLogin.bounceClick {
            proceedToLogin()
        }
    }

    private fun proceedToRegister() {
        AppStorage.get().hasSeenOnboarding = true
        AppStorage.get().isLoggedIn = false
        startActivity(Intent(this, RegisterActivity::class.java))
        finish()
    }

    private fun proceedToLogin() {
        AppStorage.get().hasSeenOnboarding = true
        AppStorage.get().isLoggedIn = false
        startActivity(Intent(this, LoginActivity::class.java))
        finish()
    }
```

Alur Guidance Pada Home

InAppGuidanceManager membangun overlay di atas layout dashboard, mencari target view berdasarkan resource ID, menggulir NestedScrollView bila target berada di bawah viewport, lalu menampilkan tooltip bertahap. Setelah langkah terakhir, flag hasSeenInAppGuidance disimpan melalui AppStorage. Karena flag ini lokal, status sudah melihat guidance tidak otomatis berpindah ke perangkat lain kecuali disinkronkan ke profile cloud.

State	Penyimpanan	Dampak
hasSeenOnboarding	SharedPreferences AppStorage	Menentukan apakah onboarding awal ditampilkan.
hasSeenInAppGuidance	SharedPreferences AppStorage	Menentukan apakah tooltip home ditampilkan.
currentUserId	SharedPreferences AppStorage	Mengikat kartu, saldo, transaksi, dan guidance user-scoped.
User profile	Firestore users/{uid} plus local cache	Nama, avatar, email, dan status account.

Lampiran B Kontrak Data Per Fitur

Tabel ini merangkum alur data dari tindakan user sampai lokasi penyimpanan. Kolom status dibaca sebagai kondisi source saat ini, bukan janji bahwa seluruh integrasi eksternal telah tersedia.

Fitur	Trigger	Kotlin atau JavaScript utama	Penyimpanan	Status
Login/register	Submit form atau Google account	LoginActivity.kt, RegisterActivity.kt	Firebase Auth, Firestore users, RTDB	Auth ada; password handling perlu

Fitur	Trigger	Kotlin atau JavaScript utama	Penyimpanan	Status
			users, AppStorage	diperkeras untuk produksi.
QRIS	Permission camera dan konfirmasi bayar	QrFragment.kt, AppStorage.kt	Local transaction, RTDB mirror, Firestore transaction	Camera preview ada; QR decode masih simulasi.
Customer service	Create ticket atau send message	CustomerSupportActivity.kt, BackOfficeRepository.kt	Firestore tickets dan messages	Realtime chat serta auto-reply prototype tersedia.
Maintenance	Toggle dari admin panel	admin-panel/app.js, MaintenanceActivity.kt	RTDB config/maintenance, Firestore bypass/audit, local cache	Berfungsi dengan catatan RTDB dan fallback harus konsisten.
Receipt	Open receipt dan tap share	ReceiptActivity.kt, FileShareHelper.kt	Intent/file cache; bukan cloud archive	Share image/PDF dan save gallery tersedia.
Cards	Add, freeze, block, delete, swipe deck	CardDeckHelper.kt, HomeFragment.kt	AppStorage, Firestore user subcollection, root mirror	Slider dan sync card tersedia; dedup perlu dijaga.
Onboarding	First launch atau first guidance	OnboardingActivity.kt, InAppGuidanceManager.kt	AppStorage flags dan resource drawable	UI onboarding dan guidance tersedia.

Lampiran C Catatan Produksi Dan Keamanan

Firebase Authentication harus menjadi satu-satunya tempat verifikasi password. Field password yang masih ada pada users/{uid}, RegisteredUserRecord, atau SharedPreferences harus dihapus pada migrasi production. Untuk aplikasi finansial, saldo dan transaksi tidak boleh bergantung pada client-side Long di AppStorage. Backend harus memakai Cloud Functions atau server terpercaya, Firestore transaction, validasi idempotency, dan audit immutable.

QRIS produksi memerlukan decoder payload dan integrasi payment acquiring. Customer Service produksi memerlukan role custom claims yang dikelola Admin SDK, bukan hanya seed localStorage. Maintenance harus memilih satu sumber kebenaran atau menjalankan dual-write yang teruji. Card Controls harus memastikan root mirror tidak dapat menampilkan kartu duplikat dan perubahan status tetap melewati Security Rules.

Dengan memahami batas ini, pembaca dapat membedakan tiga hal: alur UI yang sudah terlihat, alur data cloud yang sudah tersimpan, dan integrasi eksternal yang masih perlu backend resmi. Dokumentasi ini sengaja menyatakan ketiganya terpisah agar keputusan pengembangan berikutnya dapat dibuat dengan aman.

Lampiran D Intent, Menu, Dan Fragment

Lampiran ini menjelaskan tiga komponen Android yang menghubungkan layar-layar UTOMO Bank. Intent membawa pesan atau data antar-Activity, Menu mendefinisikan pilihan navigasi dari XML, dan Fragment menyediakan bagian UI yang dapat diganti di dalam satu Activity. Penjelasan mengikuti source code aktual project pada 14 Oktober 2026.

Pada arsitektur dashboard saat ini, DashboardActivity adalah host. Layout activity_dashboard.xml menyediakan fragmentContainer, SwipeRefreshLayout, dan bottomNavigation. Fragment mengisi fragmentContainer; Activity terpisah dipakai untuk alur penuh seperti TransferActivity, ExchangeActivity, ReceiptActivity, dan CustomerSupportActivity.

D.1 Apa Itu Intent

Intent adalah object pesan Android yang meminta komponen lain melakukan pekerjaan. Intent eksplisit menunjuk class internal secara langsung, sedangkan Intent implisit menyatakan aksi umum dan membiarkan Android memilih aplikasi yang dapat menanganinya. Intent juga dapat membawa extras sederhana seperti String, Boolean, Long, dan Parcelable; extras ini hanya data sementara untuk proses navigasi, bukan database.

Jenis	Contoh di project	Kegunaan
Eksplisit	Intent(requireContext(), TransferActivity::class.java)	Membuka layar internal UTOMO Bank yang class-nya sudah diketahui.
Implisit	Intent(Intent.ACTION_VIEW, Uri.parse(url))	Membuka browser atau aplikasi lain yang cocok dengan action.
Result Intent	setResult(Activity.RESULT_OK, intent)	Mengirim pilihan dari Activity anak kembali ke Activity pemanggil.
Activity Result API	registerForActivityResult(...)	Menerima hasil permission, Google Sign-In, kamera, atau gallery secara lifecycle-aware.

Intent Eksplisit Dan Extras

HomeFragment memakai Intent eksplisit untuk membuka fitur transaksi. BeneficiaryActivity juga memakai extras untuk mengisi otomatis nama penerima, nomor rekening, dan nama bank pada form transfer.

Pola aktual: membuka Activity dan mengirim data prefill.

```
val intent = Intent(requireContext(), TransferActivity::class.java).apply {
    putExtra("prefill_name", currentDetailName)
    putExtra("prefill_account", currentDetailAccount)
    putExtra("prefill_bank", currentDetailBank)
}
startActivity(intent)
```

AccountCardActivity menggunakan flag navigasi seperti open_history, open_qr, dan open_settings. DashboardActivity membaca flag tersebut melalui handleDestinationIntent, memilih fragment yang benar, kemudian mengubah selectedItemId pada BottomNavigationView. Kunci extras harus konsisten antara pengirim dan penerima; perubahan nama key di satu sisi saja membuat navigasi diam-diam jatuh ke Home.

Pola aktual: kembali ke DashboardActivity dan membuka tab History.

```
val intent = Intent(this, DashboardActivity::class.java).apply {
    putExtra("open_history", true)
}
startActivity(intent)
finish()
```

DashboardActivity.kt: menerjemahkan extras menjadi tujuan fragment.

```
private fun handleDestinationIntent(
    destIntent: Intent,
    bottomNav: BottomNavigationView
) {
    when {
        destIntent.getBooleanExtra("open_settings", false) -> {
            loadFragment(ProfileFragment())
            bottomNav.selectedItemId = R.id.bottom_setting
        }
        destIntent.getBooleanExtra("open_history", false) -> {
            loadFragment(HistoryFragment())
            bottomNav.selectedItemId = R.id.bottom_history
        }
        destIntent.getBooleanExtra("open_qr", false) -> {
            loadFragment(QrFragment())
            bottomNav.selectedItemId = R.id.bottom_qr
        }
        else -> {
            loadFragment(HomeFragment())
            bottomNav.selectedItemId = R.id.bottom_home
        }
    }
}
```

Intent Implisit Dan Berbagi Data

Main menu memiliki aksi berbagi yang memakai ACTION_VIEW untuk membuka URL. ReceiptActivity juga membuat Intent internal dengan banyak extras agar tampilan bukti transaksi dapat direnderer tanpa query ulang. Data tersebut berasal dari TransactionModel atau parameter fungsi start, lalu dibaca melalui Activity.intent.

DashboardActivity.kt: contoh Intent implisit ke browser.

```
val intentWeb = Intent(
    Intent.ACTION_VIEW,
    Uri.parse("https://elearning.poliupg.ac.id")
)
startActivity(intentWeb)
```

ReceiptActivity.kt: helper terpusat agar key extras tidak tersebar di banyak layar.

```
fun start(
    context: Context,
    title: String,
    category: String,
    amount: Long,
    recipientAccount: String = "",
    bankName: String = "UTOMO Bank"
) {
    val intent = Intent(context, ReceiptActivity::class.java).apply {
        putExtra(EXTRA_TITLE, title)
        putExtra(EXTRA_CATEGORY, category)
        putExtra(EXTRA_AMOUNT, amount)
        putExtra(EXTRA_RECIPIENT_ACCOUNT, recipientAccount)
        putExtra(EXTRA_BANK_NAME, bankName)
    }
    context.startActivity(intent)
}
```

Activity Result Dan setResult

BranchPickerActivity tidak membuka layar berikutnya setelah cabang dipilih. Activity tersebut membuat Intent tanpa class tujuan, memasukkan detail cabang, memanggil setResult(RESULT_OK, intent), lalu finish. Activity pemanggil menerima hasil melalui Activity Result API. Pola ini tepat untuk picker karena layar sebelumnya tetap menjadi pemilik form.

BranchPickerActivity.kt: mengembalikan cabang terpilih.

```
private fun confirmAndReturn(branch: BranchModel) {
    val result = Intent().apply {
        putExtra(EXTRA_BRANCH_ID, branch.id)
        putExtra(EXTRA_BRANCH_NAME, branch.name)
        putExtra(EXTRA_BRANCH_ADDRESS, branch.address)
        putExtra(EXTRA_BRANCH_FULL, "${branch.name} - ${branch.address}")
    }
    setResult(Activity.RESULT_OK, result)
    finish()
}
```

LoginActivity dan ProfileFragment menerapkan ActivityResultContracts.StartActivityForResult untuk alur Google Sign-In. QrFragment memakai RequestPermission untuk CAMERA dan GetContent untuk memilih gambar. Callback sebaiknya hanya memperbarui view ketika Fragment masih attached; jangan menyimpan reference View setelah onDestroyView.

D.2 Apa Itu Menu Android

Menu Android adalah resource XML di app/src/main/res/menu yang berisi item navigasi atau perintah. Setiap item memiliki ID resource yang menjadi kontrak dengan Kotlin. XML menentukan identitas tampilan seperti title dan icon; Activity menentukan perilakunya melalui listener atau onOptionsItemSelected. Menu tidak menyimpan data transaksi dan tidak menggantikan database.

File	Isi aktual	Pemakai
res/menu/bottom_nav_menu.xml	Home, History, QRIS, Statistics, Settings	BottomNavigationView pada DashboardActivity.
res/menu/main_menu.xml	Beranda dan Bagikan; item Logout didefinisikan di XML	onCreateOptionsMenu dan onOptionsItemSelected pada DashboardActivity.

File	Isi aktual	Pemakai
res/menu/nav_menu.xml	Beranda, Exchange, Statistics, Language, Profil, Notifikasi, Logout	NavigationView drawer pada DashboardActivity.

Bottom Navigation

bottom_nav_menu.xml memakai lima ID: bottom_home, bottom_history, bottom_qr, bottom_statistics, dan bottom_setting. DashboardActivity memasang setOnItemSelectedListener. Empat pilihan menampilkan fragment atau membuka StatisticsActivity; QRIS juga dapat dipilih melalui FAB QR. Karena QRIS merupakan item menu dengan icon transparan, visual tombol QRIS utama berasal dari fabQr di layout activity_dashboard.xml, sedangkan item bottom_qr tetap dipakai sebagai tujuan navigasi.

DashboardActivity.kt: listener bottom navigation aktual.

```
bottomNavigation.setOnItemSelectedListener { item ->
    var selectedFragment: Fragment? = null
    when (item.itemId) {
        R.id.bottom_home -> selectedFragment = HomeFragment()
        R.id.bottom_history -> selectedFragment = HistoryFragment()
        R.id.bottom_qr -> selectedFragment = QrFragment()
        R.id.bottom_statistics ->
            startActivity(Intent(this, StatisticsActivity::class.java))
        R.id.bottom_setting -> selectedFragment = ProfileFragment()
    }
    if (selectedFragment != null) loadFragment(selectedFragment)
    true
}
```

bottom_nav_menu.xml: item menu menjadi ID yang dibaca Kotlin.

```
<item
    android:id="@+id/bottom_history"
    android:icon="@drawable/ic_history"
    android:title="@string/nav_history" />
<item
    android:id="@+id/bottom_qr"
    android:icon="@android:color/transparent"
    android:title="" />
```

Options Menu Dan Navigation Drawer

Options menu dipasang ketika Android memanggil onCreateOptionsMenu. Saat item dipilih, DashboardActivity menerima callback onOptionsItemSelected dan mencocokkan item.itemId. Drawer memakai NavigationView dengan nav_menu.xml; callback navView.setNavigationItemSelectedListener menangani beranda, exchange, statistik, bahasa, profil, notifikasi, dan logout. Jadi ada tiga lapis navigasi yang berbeda: bottom navigation untuk area utama, options menu untuk perintah tambahan, dan drawer untuk daftar fitur yang lebih luas.

DashboardActivity.kt: menu toolbar/options.

```
override fun onCreateOptionsMenu(menu: Menu?): Boolean {
    menuInflater.inflate(R.menu.main_menu, menu)
    return true
}

override fun onOptionsItemSelected(item: MenuItem): Boolean {
    return when (item.itemId) {
        R.id.menu_beranda -> {
            loadFragment(HomeFragment())
            true
        }
        R.id.menu_bagikan -> {
            startActivity(Intent(Intent.ACTION_VIEW,
                Uri.parse("https://elearning.poliupg.ac.id")))
            true
        }
        else -> super.onOptionsItemSelected(item)
    }
}
```

D.3 Apa Itu Fragment

Fragment adalah komponen UI yang hidup di dalam Activity. Fragment memiliki lifecycle sendiri, dapat dipasang, diganti, dan dilepas oleh FragmentManager. Fragment cocok untuk layar yang berbagi shell Activity, misalnya dashboard dengan bottom navigation. Activity tetap cocok untuk alur mandiri yang memiliki form, toolbar, back stack, atau hasil transaksi sendiri.

Fragment	Tanggung jawab aktual	Akses lanjutan
HomeFragment	Card deck, saldo, quick actions, transaksi ringkas, dan shortcut fitur.	Membuka AccountCardActivity, TransferActivity, ReceiptActivity, dan fitur lain.
HistoryFragment	Pencarian, filter, total income/expense, dan daftar transaksi.	Membuka StatisticsActivity atau ReceiptActivity saat item dipilih.
QrFragment	Preview CameraX, permission kamera, gallery, dan pembayaran QRIS prototype.	Mengubah saldo/transaksi melalui AppStorage setelah konfirmasi.
ProfileFragment	Profile, bahasa, dark mode, Google linking, customer support, dan logout.	Membuka Activity atau DialogFragment sesuai fitur.

Inflate View Dan Lifecycle

onCreateView bertugas membuat atau meng-inflate layout Fragment. onViewCreated dipakai untuk findViewById dan memasang listener setelah View tersedia. onResume cocok untuk menyegarkan data ketika user kembali dari Activity lain. View Fragment dapat dihancurkan sementara walaupun object Fragment masih ada; karena itu reference View atau binding harus dilepas bila memakai ViewBinding. Kode project saat ini memakai findViewById dan sebagian besar melakukan refresh pada onResume.

HomeFragment.kt: pembuatan view dan inisialisasi layar.

```
override fun onCreateView(
    inflater: LayoutInflater,
    container: ViewGroup?,
    savedInstanceState: Bundle?
): View? {
    return inflater.inflate(R.layout.fragment_home, container, false)
}

override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    greetingView = view.findViewById(R.id.homeGreeting)
    setupCardDeck(view)
    refreshUserData()
    renderQuickHistory()
}
```

FragmentManager Dan Pergantian Layar

DashboardActivity mengganti isi fragmentContainer melalui supportFragmentManager. replace mempertahankan Activity dan bottom navigation, lalu membuat View Fragment baru. Karena transaksi ini tidak memakai addToBackStack, perpindahan tab tidak menumpuk semua tab ke tombol Back. Data bisnis tetap diambil dari AppStorage atau Firestore ketika Fragment dirender ulang; Fragment bukan tempat penyimpanan permanen.

DashboardActivity.kt: host FragmentManager.

```
private fun loadFragment(fragment: Fragment) {
    supportFragmentManager.beginTransaction()
        .replace(R.id.fragmentContainer, fragment)
        .commit()
}
```

HistoryFragment.kt: refresh data ketika user kembali ke tab History.

```
override fun onResume() {
    super.onResume()
    applyFiltersAndRender()
}
```

Hubungan Fragment ke Activity menggunakan requireContext atau requireActivity hanya ketika Fragment sedang attached. Pada callback yang tertunda, gunakan activity? atau isAdded sebelum mengaksesnya. Ini mencegah IllegalStateException ketika user cepat berpindah layar.

D.4 Alur Navigasi End-to-End

1. User mengetuk quick action pada HomeFragment, misalnya Transfer.

2. HomeFragment membuat Intent eksplisit menuju TransferActivity dan memanggil startActivity.

3. TransferActivity membaca input, memvalidasi kartu, saldo, penerima, dan nominal.

4. Jika berhasil, transaksi masuk ke AppStorage lalu disinkronkan ke Firestore sesuai user UID; layar receipt dibuka memakai ReceiptActivity.start atau startFromTransaction.

5. User menekan Back atau membuka tujuan Dashboard lagi. DashboardActivity menangani onNewIntent atau memulai HomeFragment/HistoryFragment sesuai extras.

6. Jika user memilih History dari bottom navigation, DashboardActivity menjalankan loadFragment(HistoryFragment()). HistoryFragment membaca transaksi terbaru dan menampilkan receipt ketika item diketuk.

Tiga pola perpindahan layar yang dipakai project.

```
// Fragment -> Activity
startActivity(Intent(requireContext(), TransferActivity::class.java))

// Activity -> Fragment pada Dashboard
putExtra("open_history", true)

// Fragment -> Fragment tanpa Activity baru
requireActivity().supportFragmentManager.beginTransaction()
    .replace(R.id.fragmentContainer, HistoryFragment())
    .commit()
```

Intent extras hanya membawa konteks navigasi atau prefill. Saldo, kartu, transaksi, dan ticket tidak boleh dianggap sudah tersimpan hanya karena extras berhasil dikirim. Penyimpanan permanen dilakukan oleh AppStorage sebagai cache/state runtime dan repository Firestore sebagai cloud source sesuai bagian arsitektur data pada dokumentasi utama.

D.5 Peta File Dan Kontrak ID

Bagian	Lokasi	Kontrak yang harus konsisten
Host dashboard	app/src/main/java/.../DashboardActivity.kt	R.id.fragmentContainer, R.id.bottomNavigation, dan destination extras.
Layout host	app/src/main/res/layout/activity_dashboard.xml	ID container, SwipeRefreshLayout, bottomNavContainer, dan fabQr.
Menu tab	app/src/main/res/menu/bottom_nav_menu.xml	bottom_home, bottom_history, bottom_qr, bottom_statistics, bottom_setting.
Menu toolbar	app/src/main/res/menu/main_menu.xml	menu_beranda, menu_bagikan, menu_logout.
Menu drawer	app/src/main/res/menu/nav_menu.xml	nav_beranda, nav_exchange, nav_statistics, nav_language, nav_profil, nav_notifikasi, nav_logout.
Isi tab	HomeFragment.kt, HistoryFragment.kt, QrFragment.kt, ProfileFragment.kt	Layout fragment dan listener view harus sesuai ID XML.

Titik rawan paling umum adalah ID XML yang tidak sama dengan R.id di Kotlin, item menu ditambahkan tetapi tidak ditangani pada when, atau Activity tujuan belum terdaftar di AndroidManifest.xml. Ketiganya dapat membuat item tampak ada tetapi tidak melakukan apa-apa.

D.6 Kesalahan Umum Dan Cara Memeriksa

Gejala	Penyebab yang mungkin	Pemeriksaan
Klik menu kembali ke Home	Extra key salah atau handleDestinationIntent tidak membaca key.	Bandingkan key putExtra dengan getBooleanExtra dan cek log intent.
Bottom nav terlihat tetapi tab tidak berganti	ID item tidak sama atau listener tidak mengembalikan true.	Cek bottom_nav_menu.xml dan when item.itemId.

Gejala	Penyebab yang mungkin	Pemeriksaan
Layar putih atau view tidak muncul	Layout Fragment salah, inflate tidak dipanggil, atau resource ID tidak ada.	Periksa onCreateView, nama layout, dan Logcat InflateException.
Crash setelah kembali dari layar lain	Callback mengakses requireActivity/requireContext saat Fragment sudah detached.	Gunakan isAdded, activity?, dan hentikan callback pada lifecycle yang sesuai.
Data lama muncul setelah tab dibuka	Fragment tidak refresh ketika onResume atau cache tidak disinkronkan.	Panggil repository/AppStorage refresh pada onResume dan tampilkan state loading.
Activity picker tidak mengembalikan pilihan	Pemanggil tidak mendaftarkan launcher atau result code bukan RESULT_OK.	Periksa registerForActivityResult dan setResult(Activity.RESULT_OK, result).
Menu XML ada tetapi tidak tampil di toolbar	Menu belum di-inflate atau Toolbar belum terhubung.	Periksa onCreateOptionsMenu, setSupportActionBar, dan showAsAction bila diperlukan.

Kesimpulan

Dalam UTOMO Bank, Intent menghubungkan alur antar layar, Menu menyediakan kontrak pilihan navigasi, dan Fragment membagi dashboard menjadi tab yang dapat diganti tanpa membuat Activity baru. Ketiganya mengatur tampilan dan navigasi, bukan menggantikan Firebase. SharedPreferences hanya cocok untuk session serta preferensi yang memang disepakati; data kartu, saldo, transaksi, ticket, dan profil tetap harus melewati repository cloud dan aturan keamanan Firebase.

Lampiran E Kebijakan Penyimpanan Firebase Final

Setelah audit source code, kebijakan penyimpanan aplikasi ditetapkan sebagai berikut: SharedPreferences bukan database bisnis. Ia hanya menyimpan session perangkat, pilihan dark mode atau light mode, dan bahasa. Semua data yang harus tetap ada setelah logout, restart, reinstall, atau login dari perangkat lain harus berasal dari Firebase.

Kelompok data	Sumber kebenaran	Lokasi Firebase
Session perangkat	SharedPreferences dan Firebase Auth session	Firebase Authentication UID; SharedPreferences hanya flag/session pointer.
Dark mode / light mode	SharedPreferences	Preferensi tampilan lokal perangkat.
Bahasa	SharedPreferences melalui LocaleHelper	Preferensi tampilan lokal perangkat.
Profil dan keamanan	Cloud Firestore	users/{uid}: name, email, phone, avatar, PIN state, biometric state, Google link, status.
Saldo dan kurs	Cloud Firestore	users/{uid}: debitBalance, savingsBalance, creditCardBalance, creditCardLimit, monthlySpendingLimit, currencyBalances.
Kartu	Cloud Firestore	users/{uid}/cards/{cardId}; mirror admin cards/{uid}_{cardId}.
Transaksi	Cloud Firestore	transactions/{transactionId} dengan userId, accountId, nominal, kategori, status, dan createdAt.
Transaksi favorit	Cloud Firestore	users/{uid}/beneficiaries/{beneficiaryId}.
Budget planner	Cloud Firestore	users/{uid}/savingGoals/{goalId}.
Customer service	Cloud Firestore	tickets/{ticketId}, messages/{messageId}, dan internalNotes/{noteId}.
Notifikasi	Cloud Firestore	users/{uid}/notifications/{notificationId}.
Cabang dan ATM	Cloud Firestore	branches/{branchId}; daftar bawaan hanya fallback tampilan jika koleksi belum diisi.
Maintenance dan bypass	Firebase RTDB + Cloud Firestore	RTDB config/maintenance untuk status cepat; Firestore config/maintenance, maintenance_bypass, dan audit_logs untuk administrasi.

Perubahan Yang Sudah Diterapkan

AppStorage sekarang menggunakan field memory sebagai cache proses dan melakukan hydration dari Firestore setelah Firebase Authentication berhasil. Cache memory tidak bertahan sebagai database lokal. SplashActivity juga

menunggu hydrateUserData sebelum membuka dashboard sehingga saldo, kartu, kurs, transaksi, beneficiary, dan saving goal tidak kembali ke nilai awal ketika aplikasi dibuka ulang.

activeSupportTicketId dipindahkan dari SharedPreferences ke memory-only karena hanya penunjuk layar yang sedang dibuka. RegisterActivity, BaseActivity, dan DashboardActivity tidak lagi menulis atau membaca SharedPreferences secara langsung untuk data bisnis; semuanya melalui AppStorage. Pemeriksaan source menunjukkan akses SharedPreferences tersisa hanya pada AppStorage dan LocaleHelper untuk session, tema, dan bahasa.

Cara Memastikan Data Masuk Firebase

Untuk memeriksa transaksi, buka Firestore > transactions dan cari field accountId atau userId sesuai Firebase Authentication UID. Untuk kartu, buka users > {uid} > cards dan cards untuk mirror admin. Untuk favorit, buka users > {uid} > beneficiaries. Untuk budget, buka users > {uid} > savingGoals. Untuk saldo dan kurs, buka dokumen users > {uid} dan lihat debitBalance, savingsBalance, serta currencyBalances.

Jika data tidak muncul, periksa tiga hal: Firebase Authentication harus memiliki currentUser, UID pada request harus sama dengan dokumen owner, dan Firestore Rules harus mengizinkan operasi tersebut. Kegagalan sinkronisasi dicatat melalui Logcat dengan tag BackOfficeRepository; tampilan lokal memory tidak boleh dianggap sebagai bukti write Firebase berhasil.

Catatan Migrasi Data Lama

Data bisnis yang pernah dibuat oleh versi lama di SharedPreferences tidak lagi dibaca sebagai sumber utama. Data lama perlu dimigrasikan satu kali ke Firestore melalui admin atau skrip migrasi berdasarkan user UID. Jangan menyalin password ke Firestore; password hanya dikelola Firebase Authentication. Setelah migrasi selesai, dokumen Firestore menjadi sumber kebenaran dan aplikasi melakukan hydration setiap login atau pembukaan session.

Batas Cache Yang Masih Diizinkan

Variabel memory seperti memoryCards, memoryTransactions, dan memoryCurrencyBalances tetap ada agar UI dapat bekerja cepat selama proses berjalan. Variabel tersebut bukan penyimpanan permanen dan akan dihapus saat user berganti atau session di-reset. File cache untuk berbagi receipt juga bersifat sementara. File avatar custom sebaiknya dipindahkan ke Cloud Storage bila fitur upload avatar digunakan sebagai data profil lintas perangkat; preset avatar tetap cukup direpresentasikan oleh nama resource di dokumen user.